

Cybernetics Oriented Programming (CYBOP)

- in Res Medicinae -

Copyright (c) 2002. Christian Heller. All rights reserved.
Distributable under the GNU FDL license.
<http://www.resmedicinae.org> christian.heller@tuxtax.de



Contents



1 Introduction

2 Classical System Design

2.1 Physical Architecture

2.2 Logical Architecture

3 CYBOP

3.1 System Modelling

3.2 Hierarchy

3.3 Ontology

3.4 Lifecycle

3.5 Linking

3.6 Advantages

4 Res Medicinae

4.1 Aims

4.2 Analysis

4.3 Design

4.4 Implementation

4.5 Vision

5 Summary and Future

6 Appendix

6.1 Abbreviations

6.2 Sources

6.3 Tools

1 Introduction

1 Introduction

2 Classical System Design

2.1 Physical Architecture

2.2 Logical Architecture

3 CYBOP

3.1 System Modelling

3.2 Hierarchy

3.3 Ontology

3.4 Lifecycle

3.5 Linking

3.6 Advantages

4 Res Medicinae

4.1 Aims

4.2 Analysis

4.3 Design

4.4 Implementation

4.5 Vision

5 Summary and Future

6 Appendix

6.1 Abbreviations

6.2 Sources

6.3 Tools

1 Introduction



Informatics/Software:

- ease our life by imitating human behaviour (machines, robots)
- build abstract models of real world (nature shows solutions)
- not done well in today's software systems

- we need **systematics** to sort information/build up knowledge
- Platon-Aristotle-Alexander the Great: **category**
- modern programming languages provide not only methods but have introduced the concept of **classes** for high level abstraction
- many different **frameworks** (class libraries)
- application of **design patterns** (constellations of classes)
- still redundant parts, different/overlapping hierarchies

2 Classical System Design

1 Introduction

2 Classical System Design

2.1 Physical Architecture

2.2 Logical Architecture

3 CYBOP

3.1 System Modelling

3.2 Hierarchy

3.3 Ontology

3.4 Lifecycle

3.5 Linking

3.6 Advantages

4 Res Medicinae

4.1 Aims

4.2 Analysis

4.3 Design

4.4 Implementation

4.5 Vision

5 Summary and Future

6 Appendix

6.1 Abbreviations

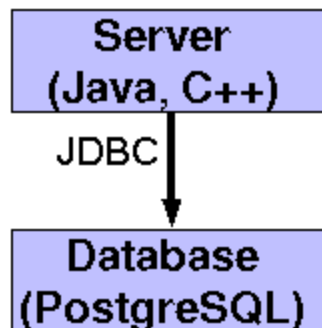
6.2 Sources

6.3 Tools

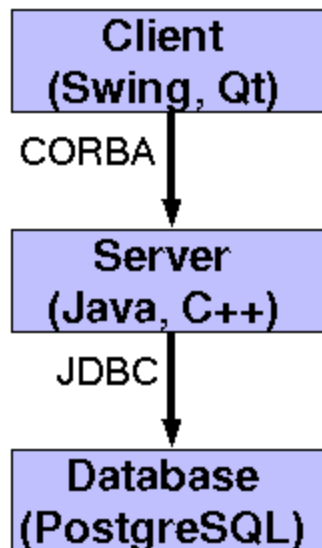
2.1 Physical Architecture: 1 Tier

Server
(Java, C++)

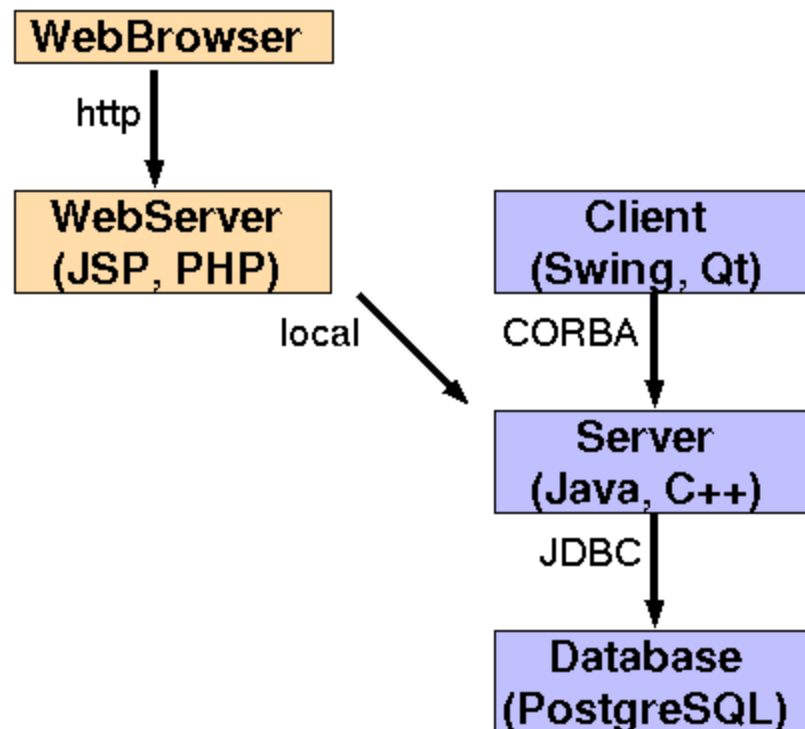
2.1 Physical Architecture: 2 Tiers



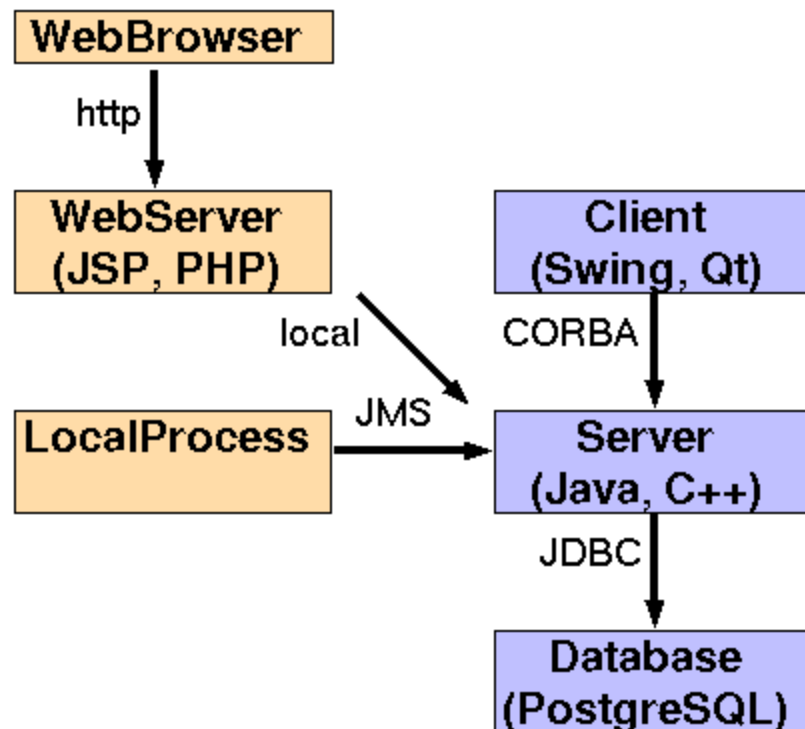
2.1 Physical Architecture: 3 Tiers



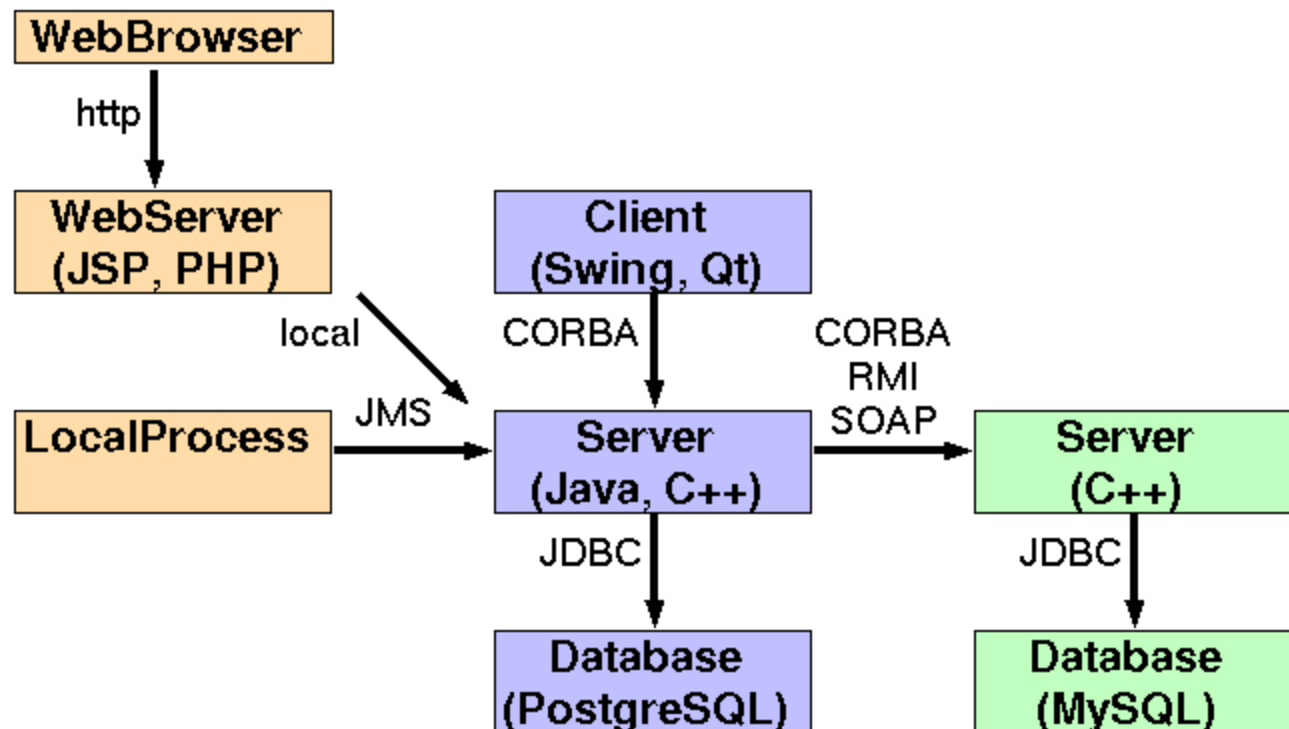
2.1 Physical Architecture: Web



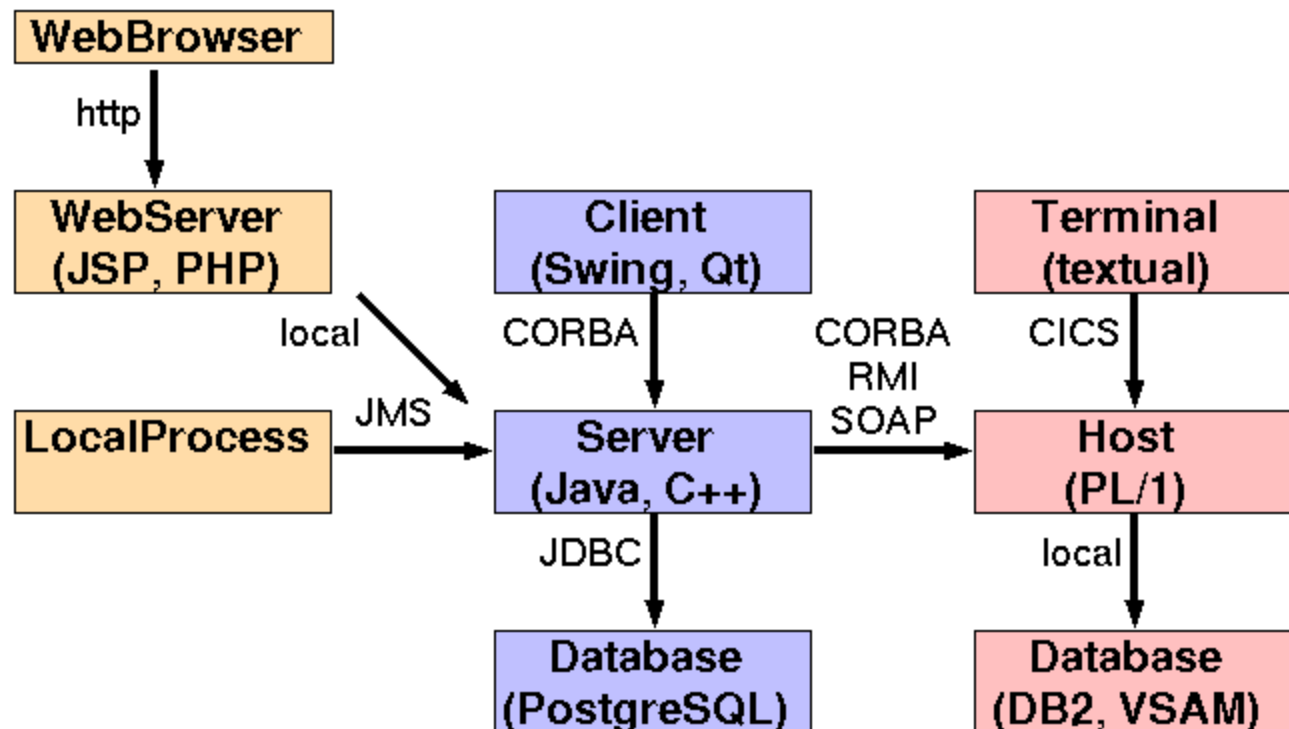
2.1 Physical Architecture: Local



2.1 Physical Architecture: Remote

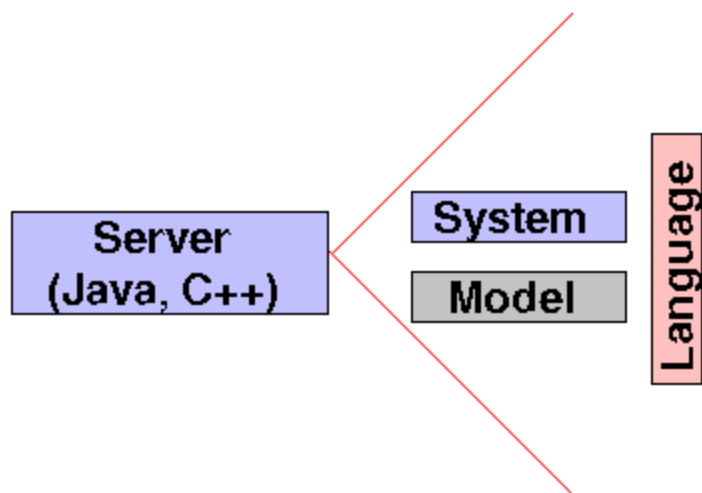


2.1 Physical Architecture: Legacy



2.1 Physical Architecture: Layers

Systems (Processes)
consist of logical layers.



2.2 Logical Architecture

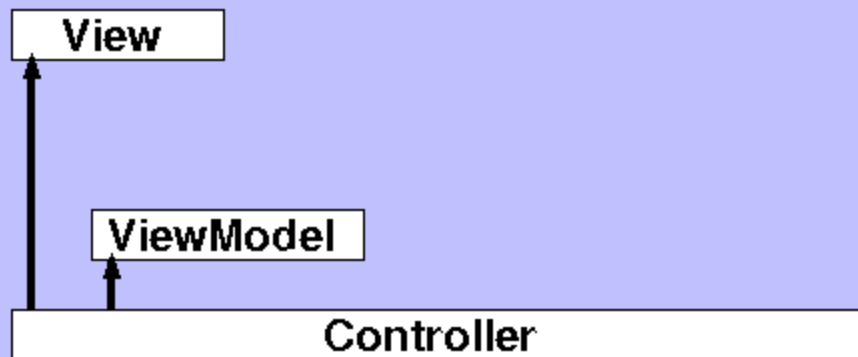
System

The diagram illustrates the Logical Architecture of a system. It consists of three main components arranged in a 2x2 grid, with the bottom-right cell being empty. The top-left component is a large blue rectangle labeled 'System'. The bottom-left component is a smaller grey rectangle labeled 'Model'. The bottom-right component is a smaller green rectangle labeled 'Remote'. All three components are contained within a larger white rectangle that has a black border.

Model

Remote

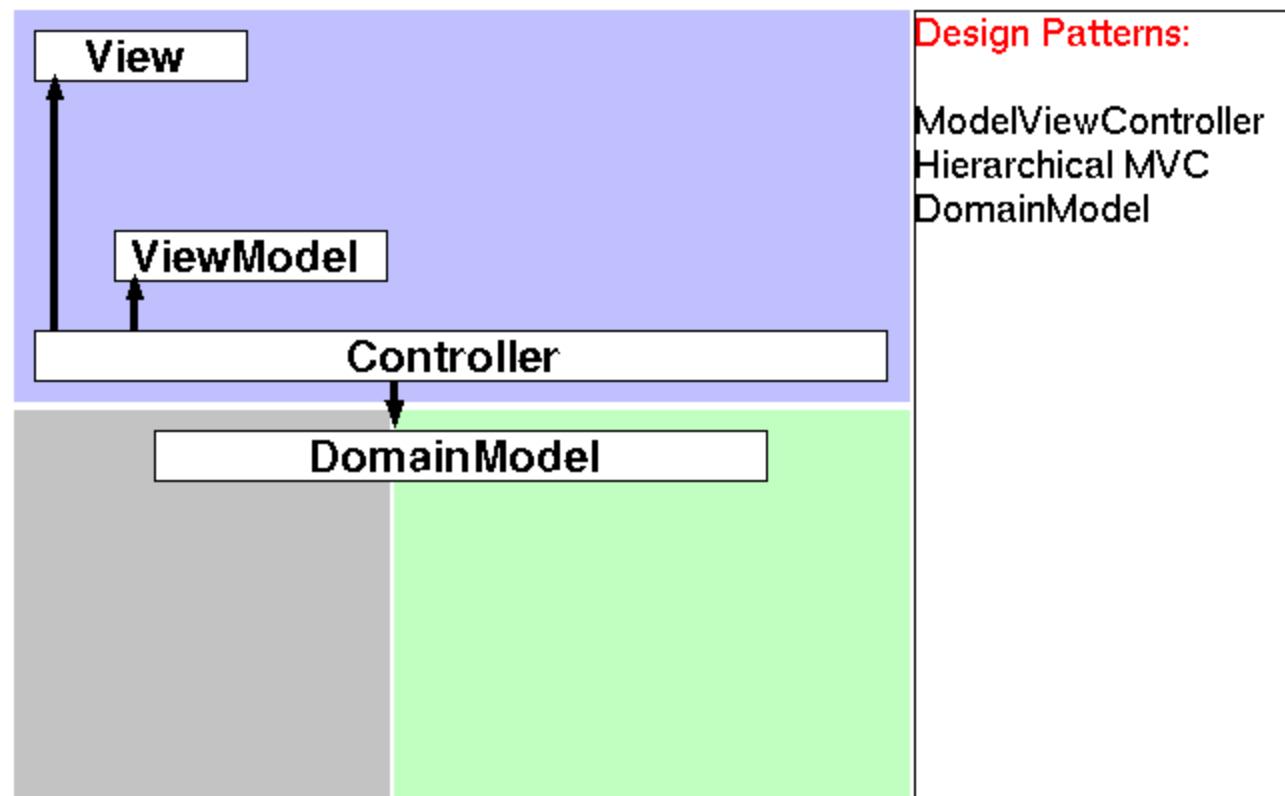
2.2 Logical Architecture



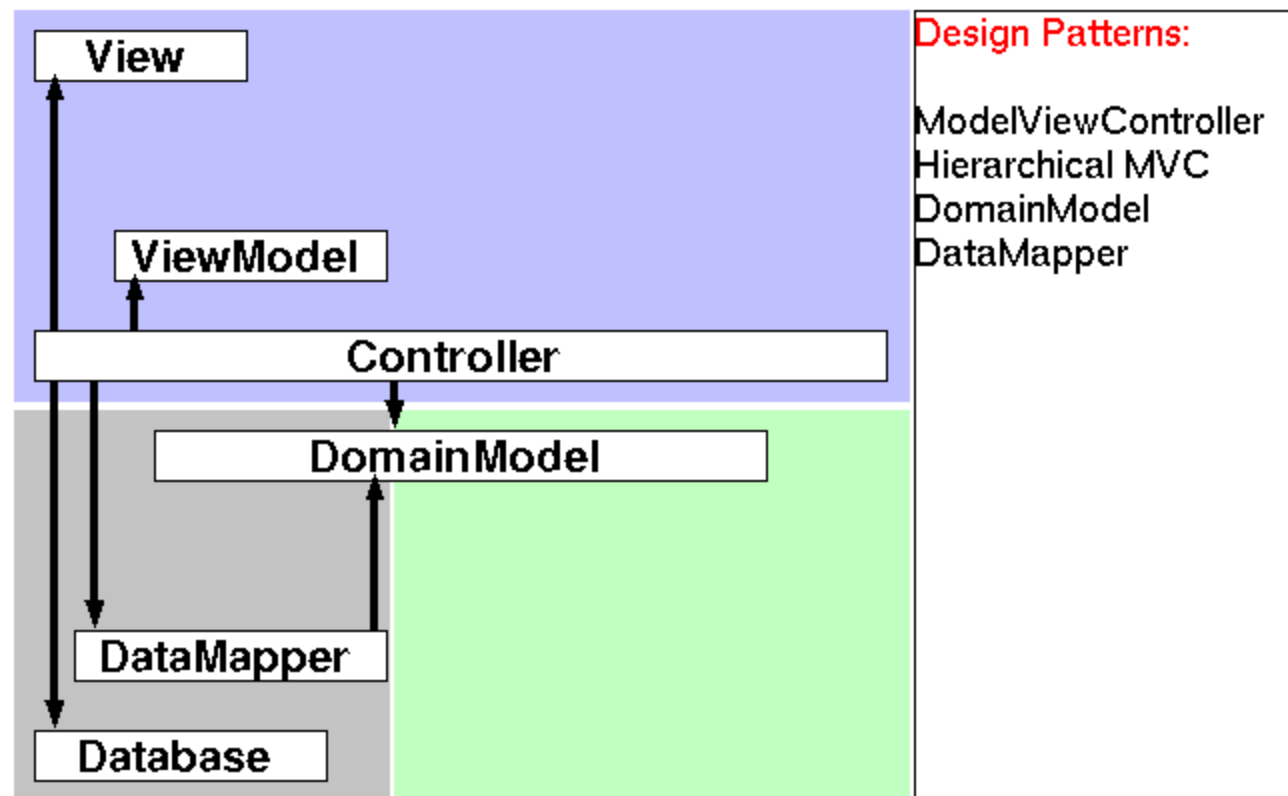
Design Patterns:

ModelViewController
Hierarchical MVC

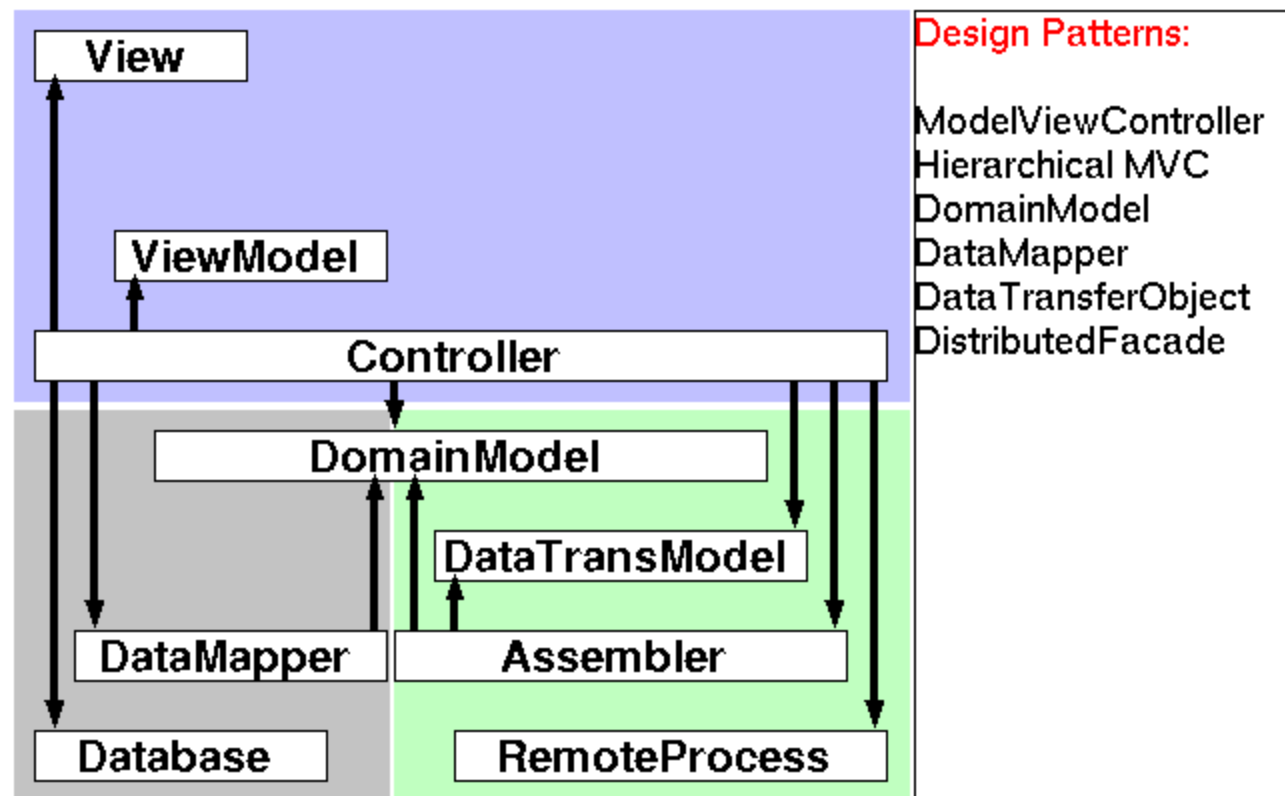
2.2 Logical Architecture



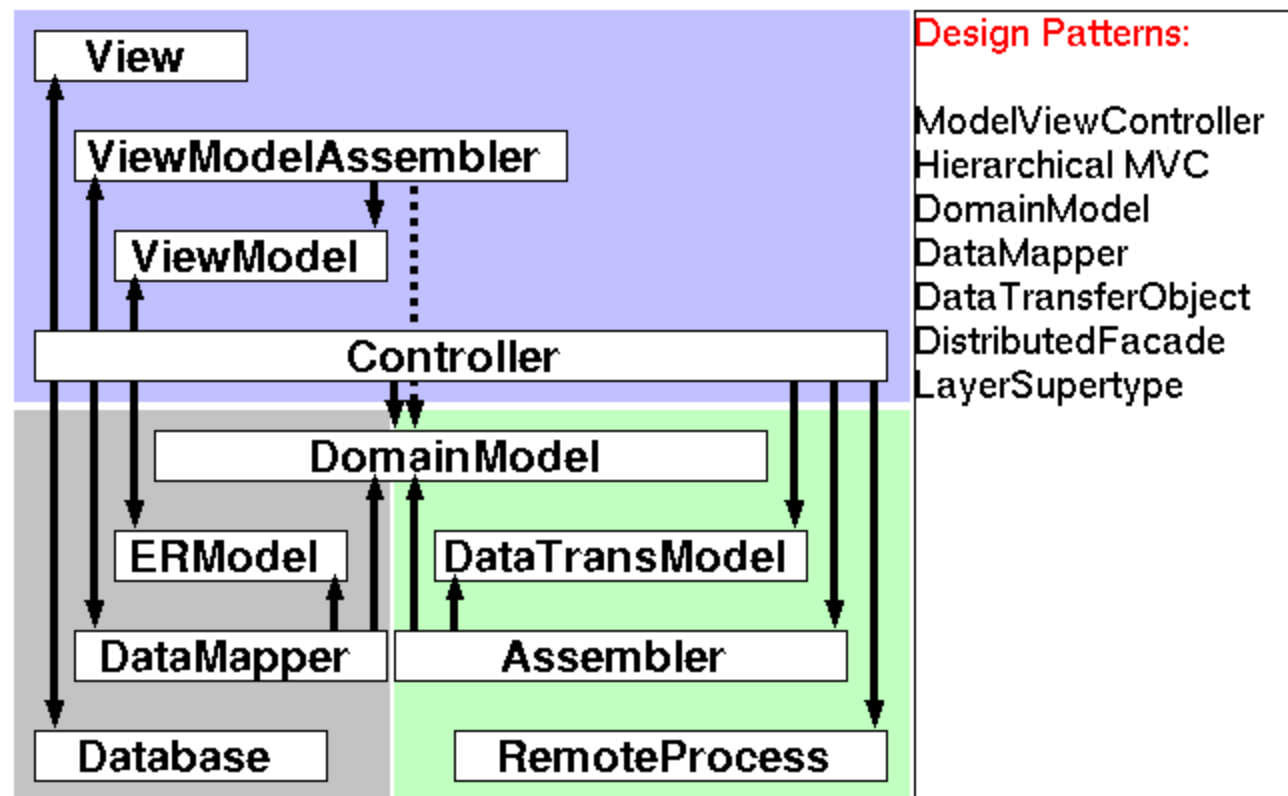
2.2 Logical Architecture



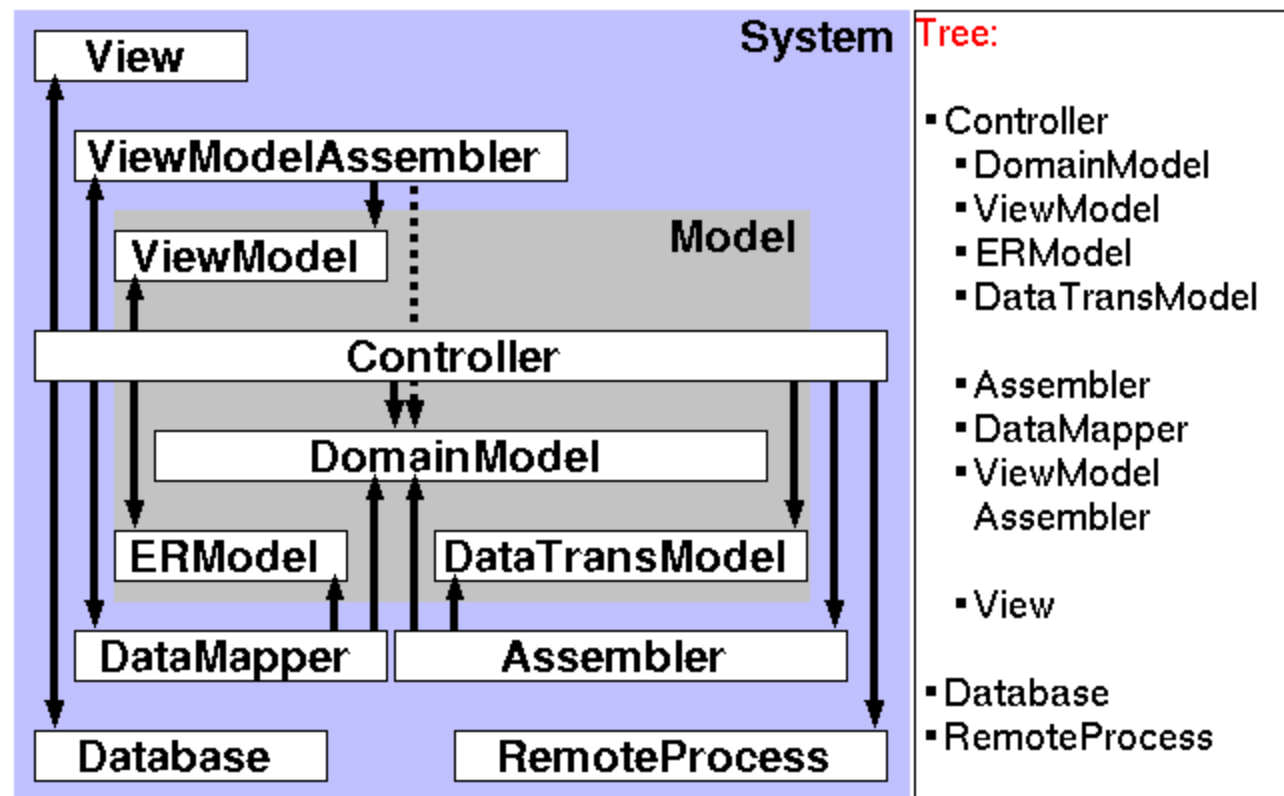
2.2 Logical Architecture



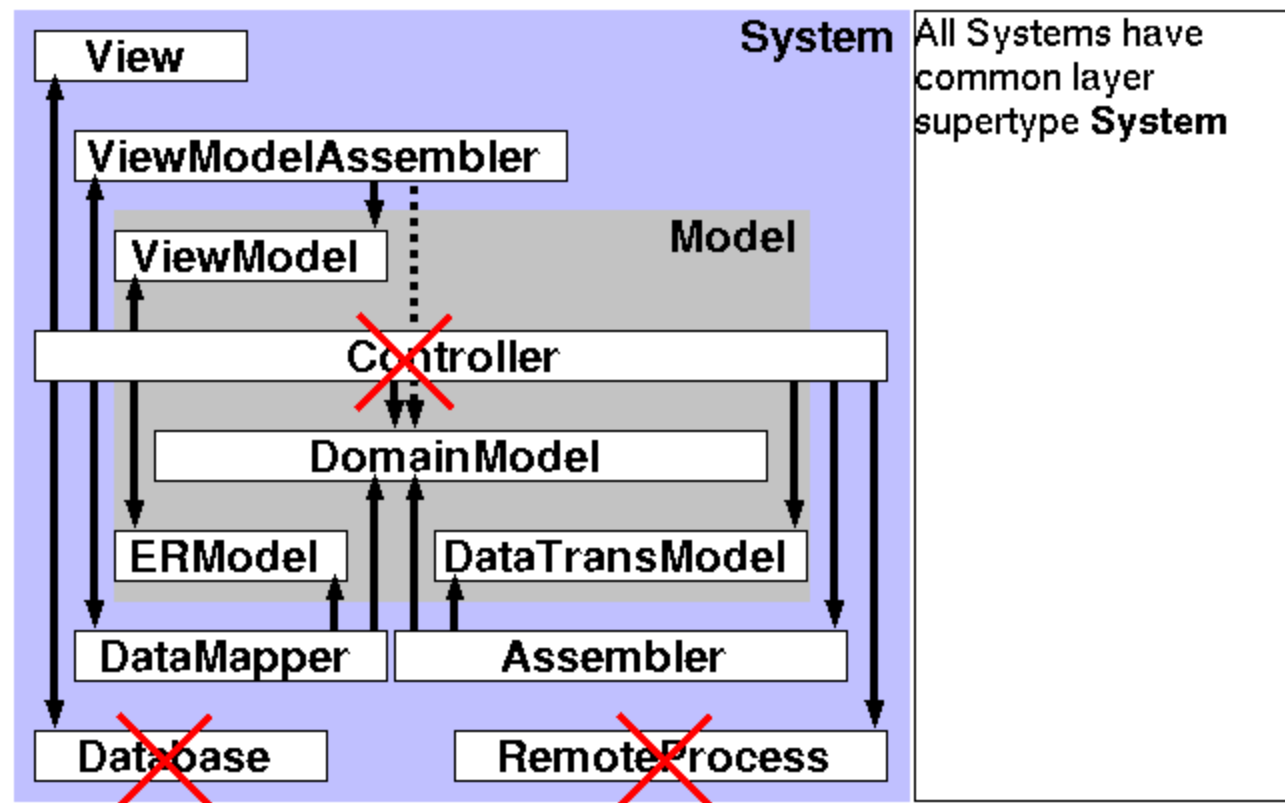
2.2 Logical Architecture



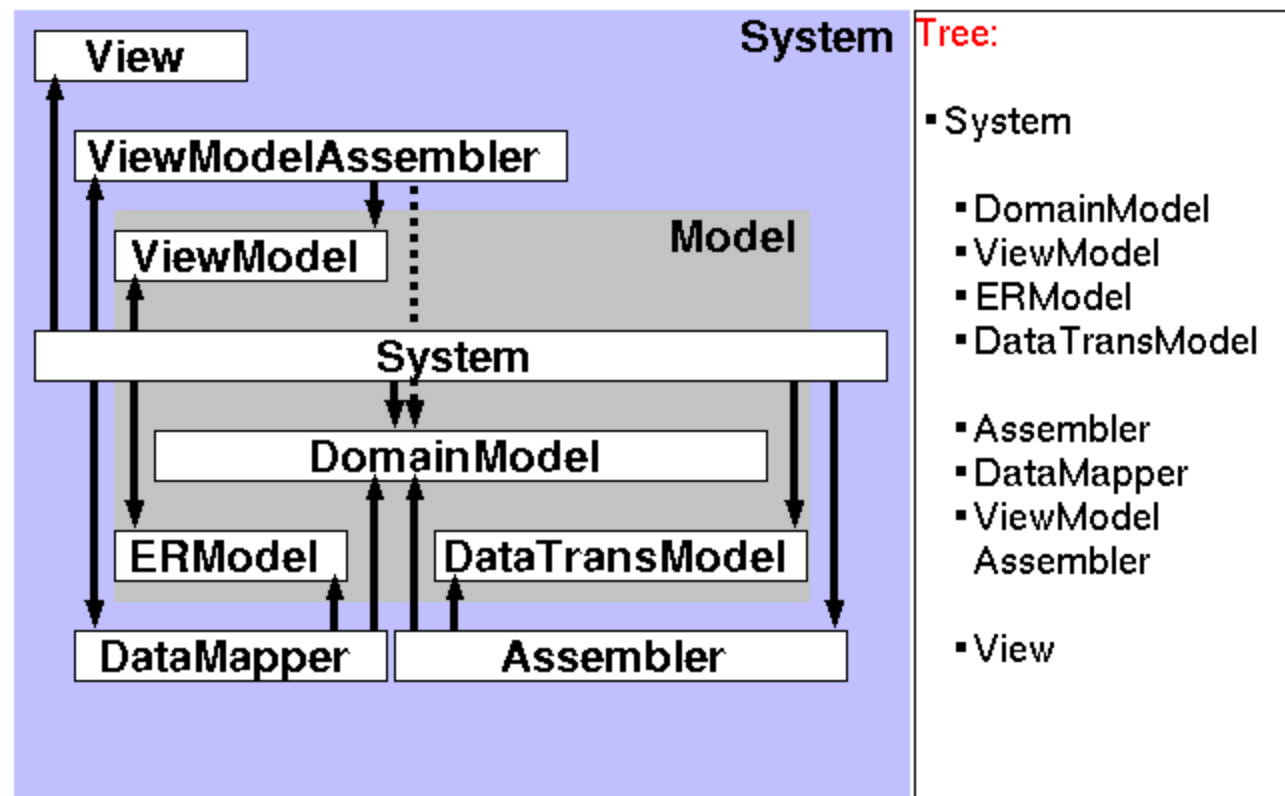
2.2 Logical Architecture



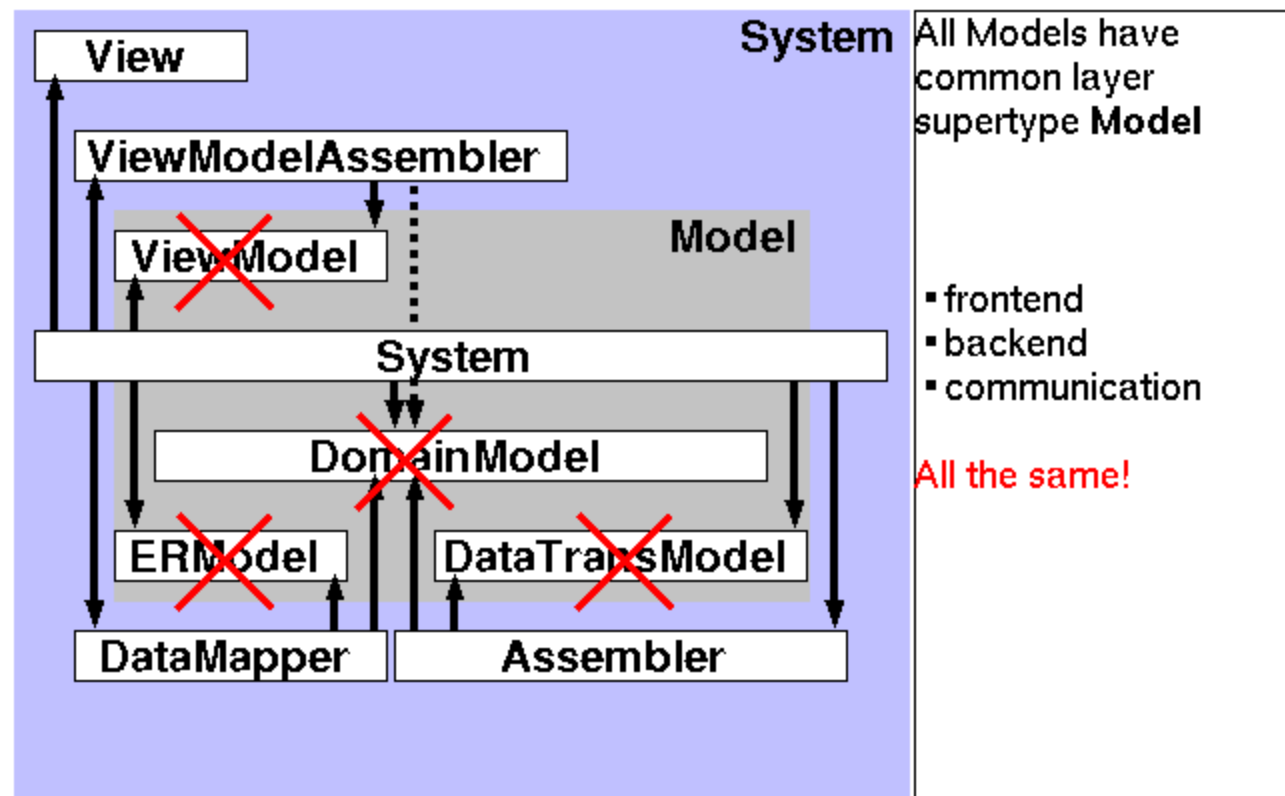
2.2 Logical Architecture



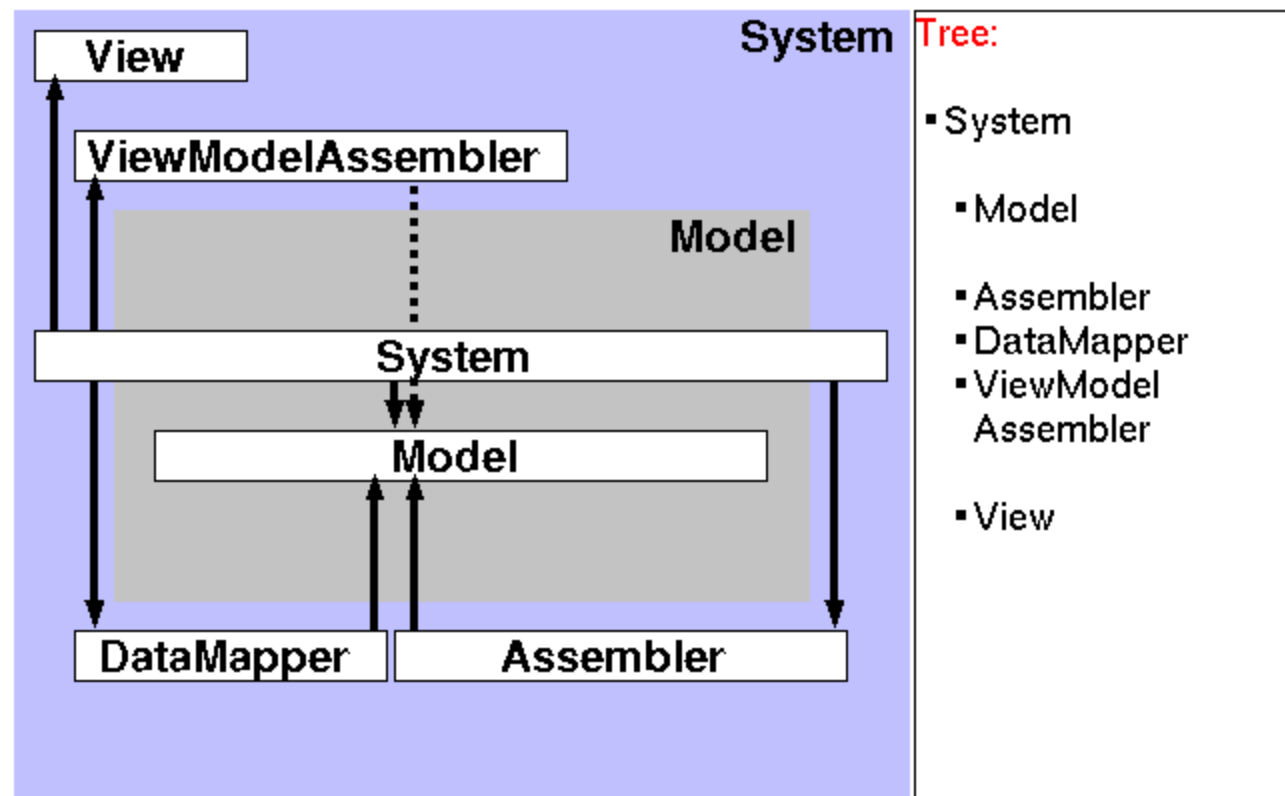
2.2 Logical Architecture



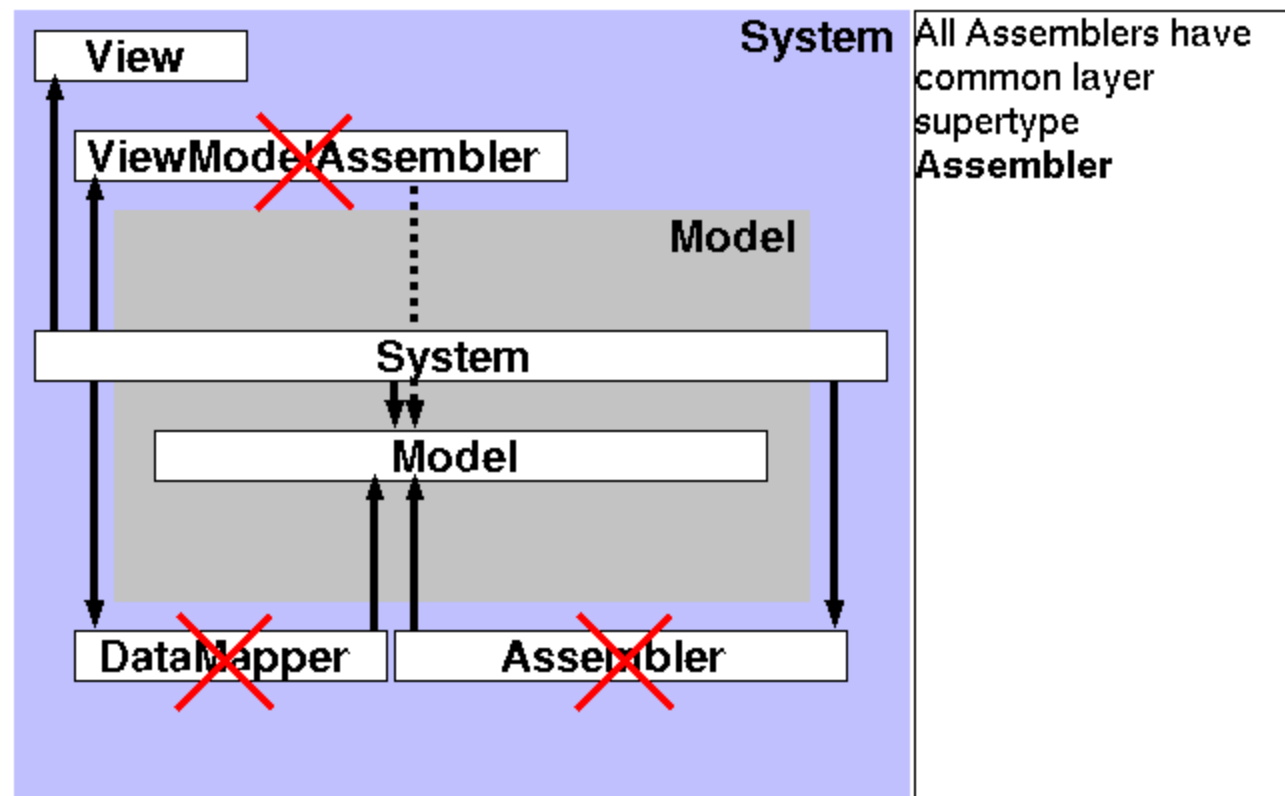
2.2 Logical Architecture



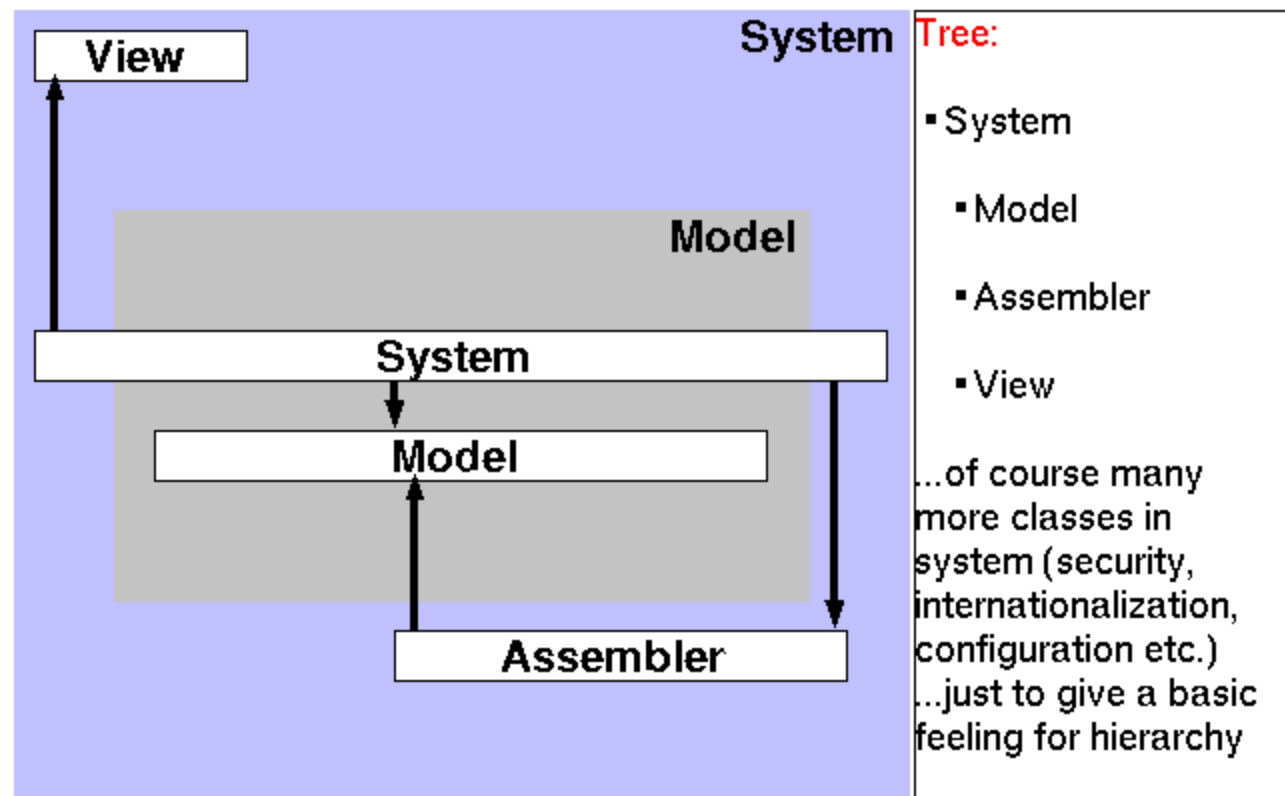
2.2 Logical Architecture



2.2 Logical Architecture



2.2 Logical Architecture



3 CYBOP

1 Introduction

2 Classical System Design

2.1 Physical Architecture

2.2 Logical Architecture

3 CYBOP

3.1 System Modelling

3.2 Hierarchy

3.3 Ontology

3.4 Lifecycle

3.5 Linking

3.6 Advantages

4 Res Medicinae

4.1 Aims

4.2 Analysis

4.3 Design

4.4 Implementation

4.5 Vision

5 Summary and Future

6 Appendix

6.1 Abbreviations

6.2 Sources

6.3 Tools

3.1 System Modelling: Kybernetik



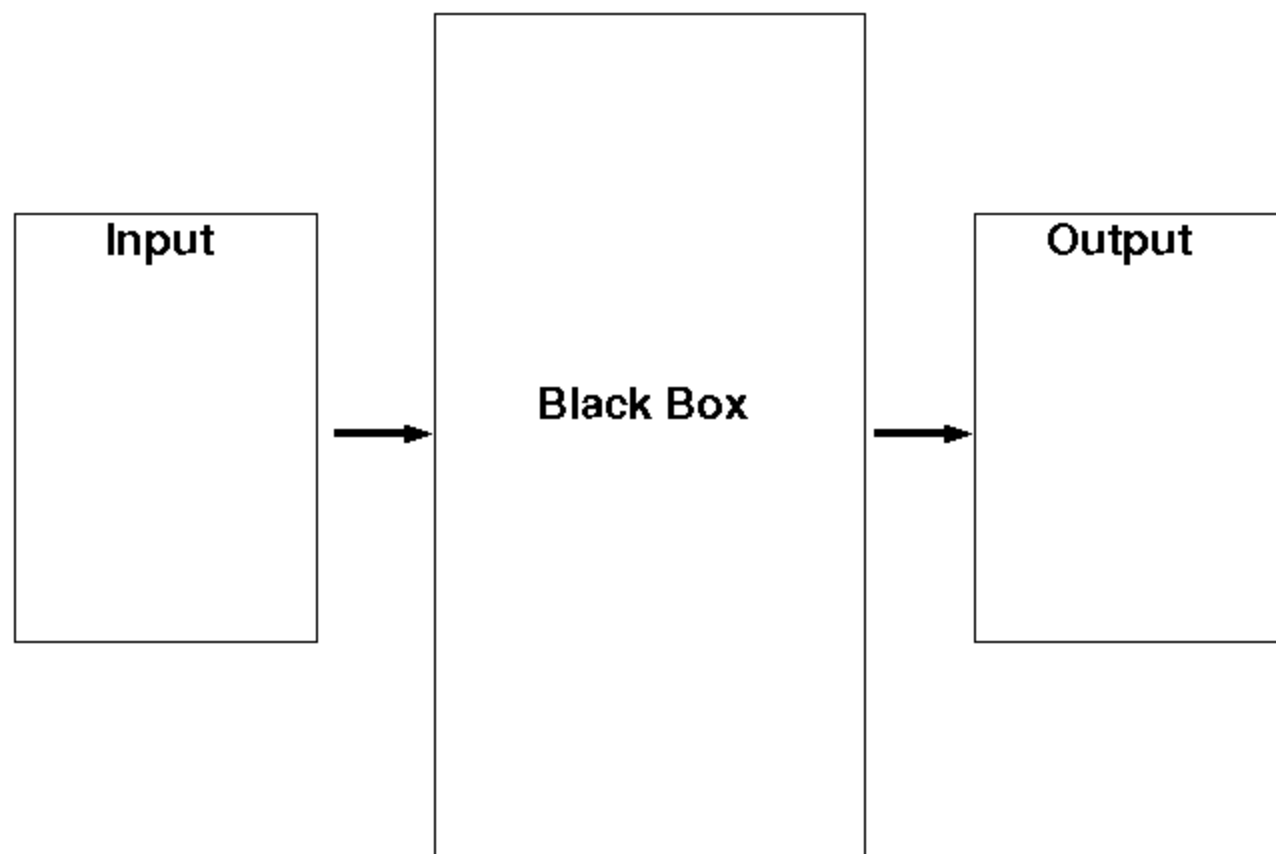
The word **Kybernetik** was formed by N. Wiener in 1948.

It is meant to be the **science of information and control** (System Engineering), no matter whether it is about living things or machines.

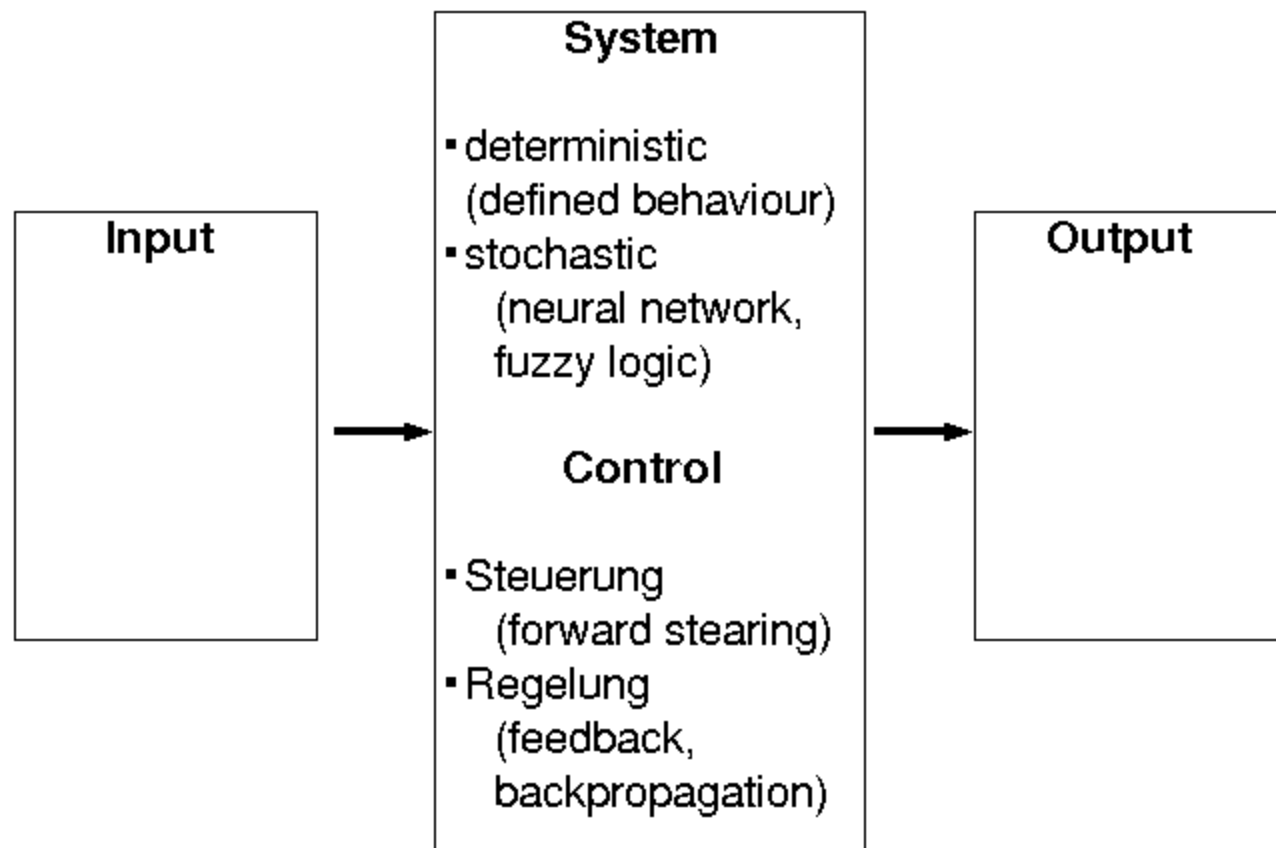
Principle:

- inspect what solutions nature offers to us
- build abstract models of these
- implement them in technics (hardware or software)

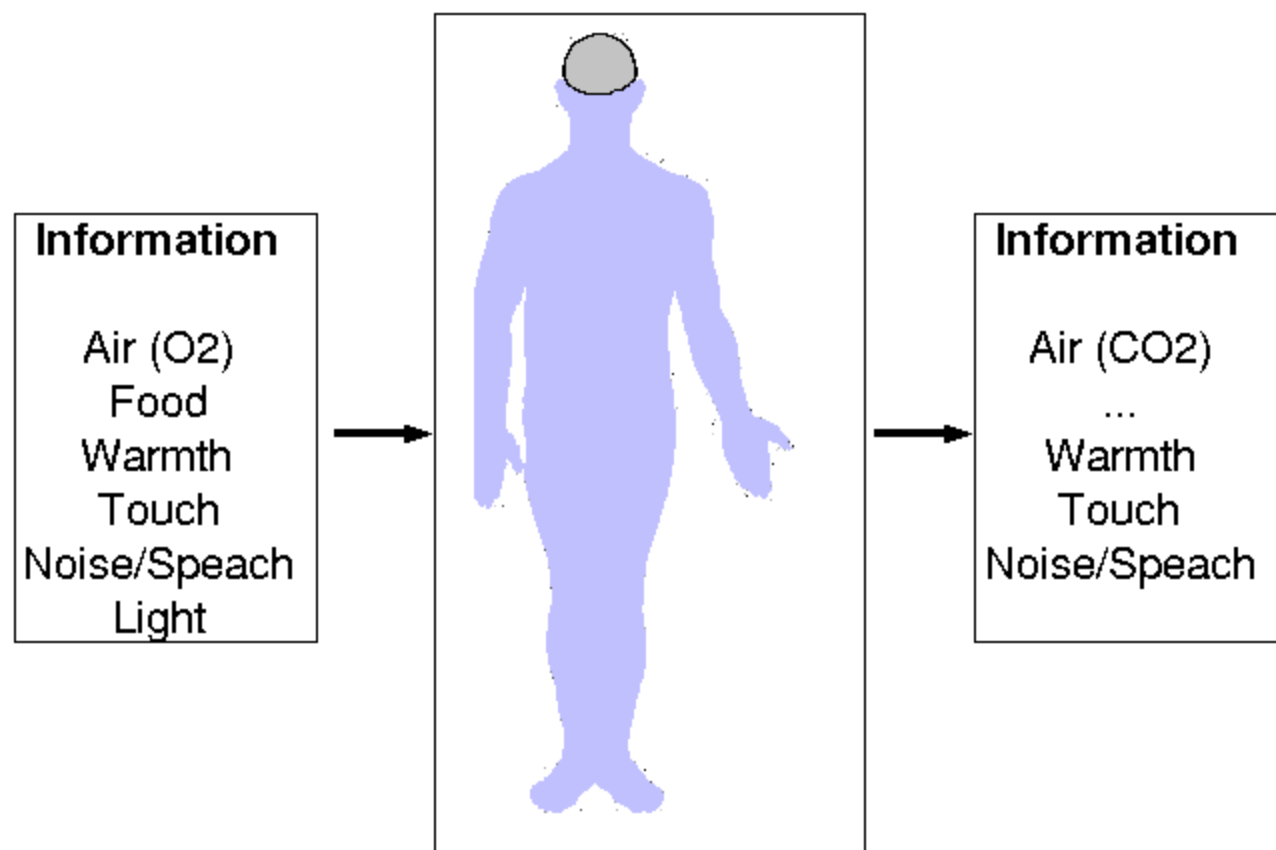
3.1 System Modelling: Black Box



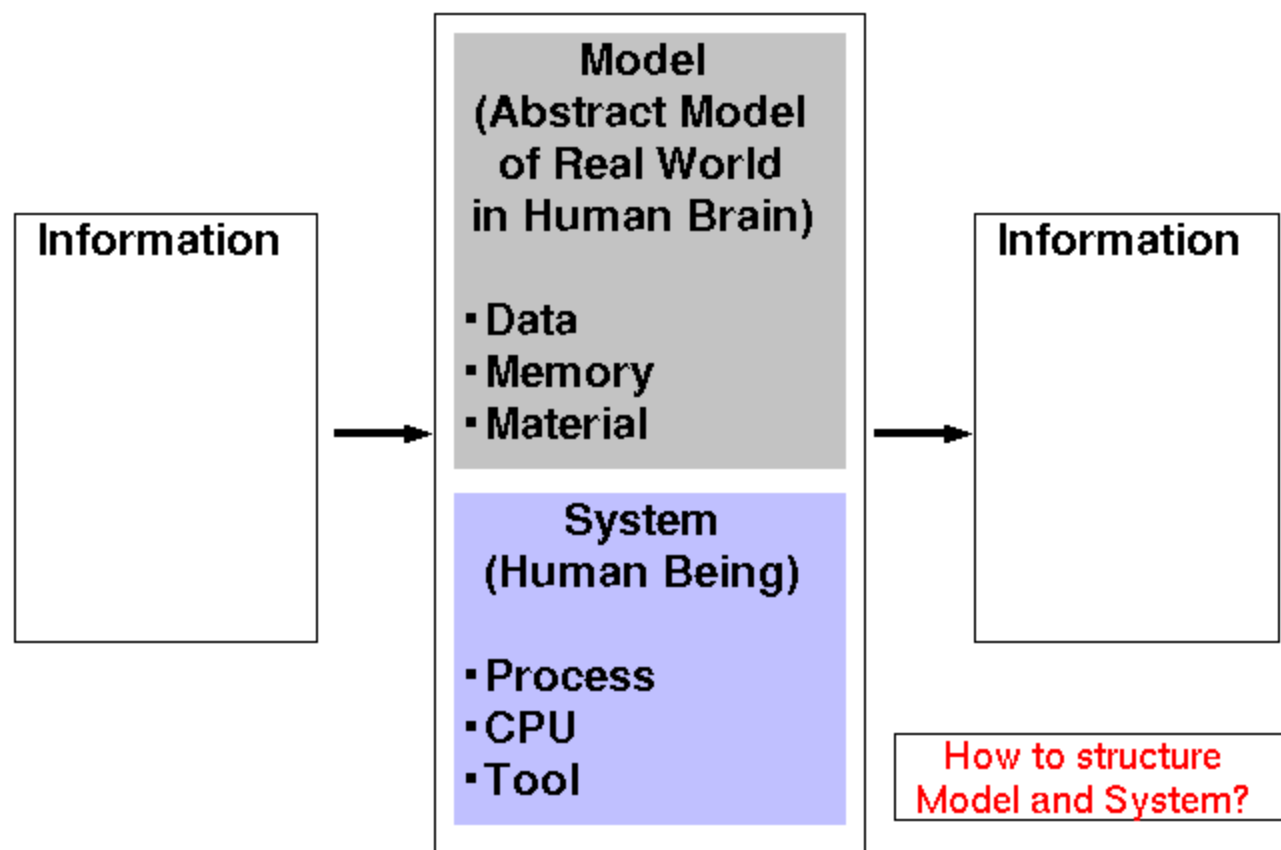
3.1 System Modelling: Control



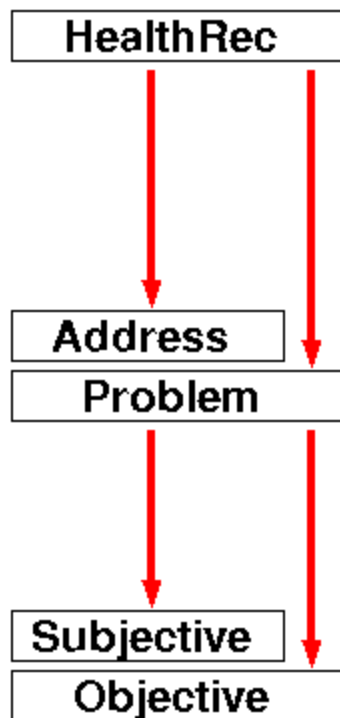
3.1 System Modelling: Human



3.1 System Modelling: Areas



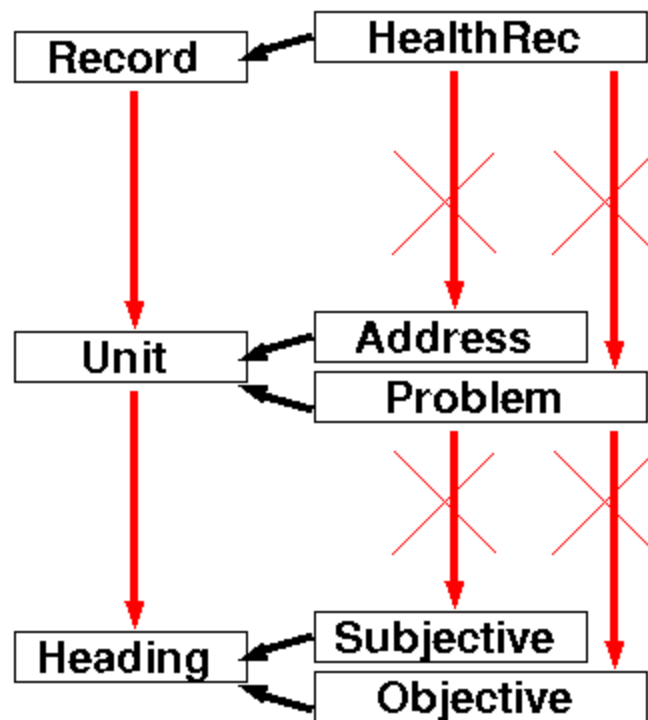
3.2 Hierarchy



Framework:

- consisting of classes with associations

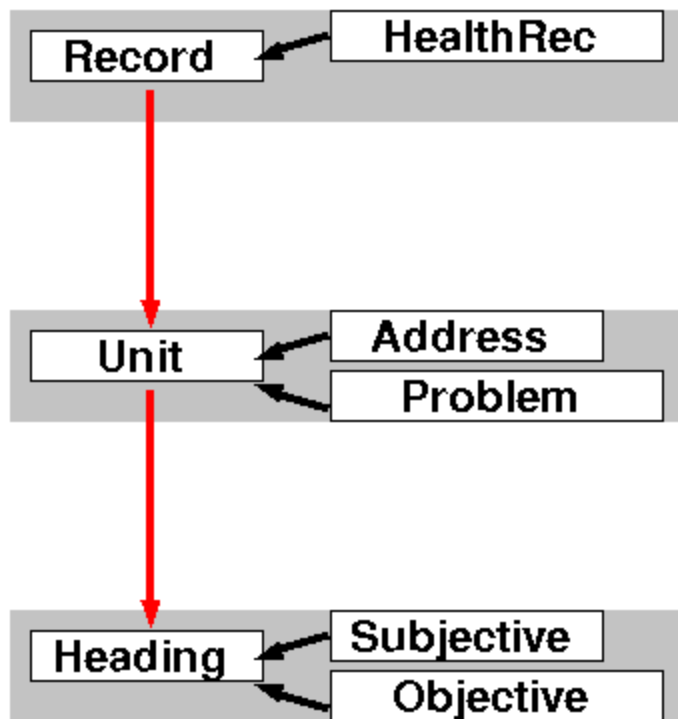
3.2 Hierarchy



Design Pattern:

- Layer
- Supertype
- summing up common properties and behaviour

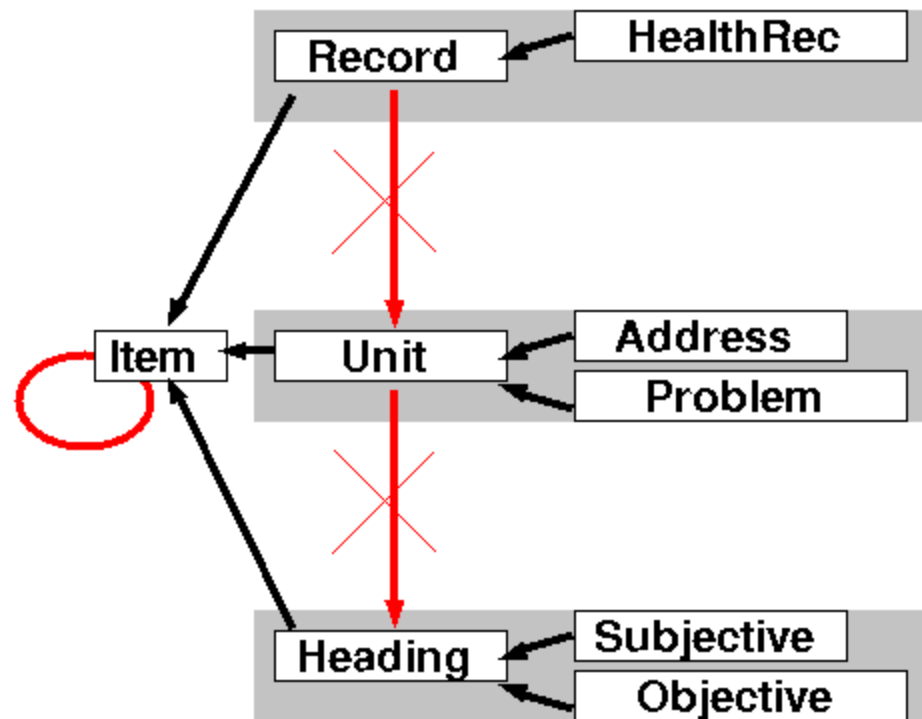
3.2 Hierarchy



Layers:

- Ontological Levels
- classifying into groups of common granularity
- no interfaces/ multiple inheritance/ redundant code
- no abstract classes

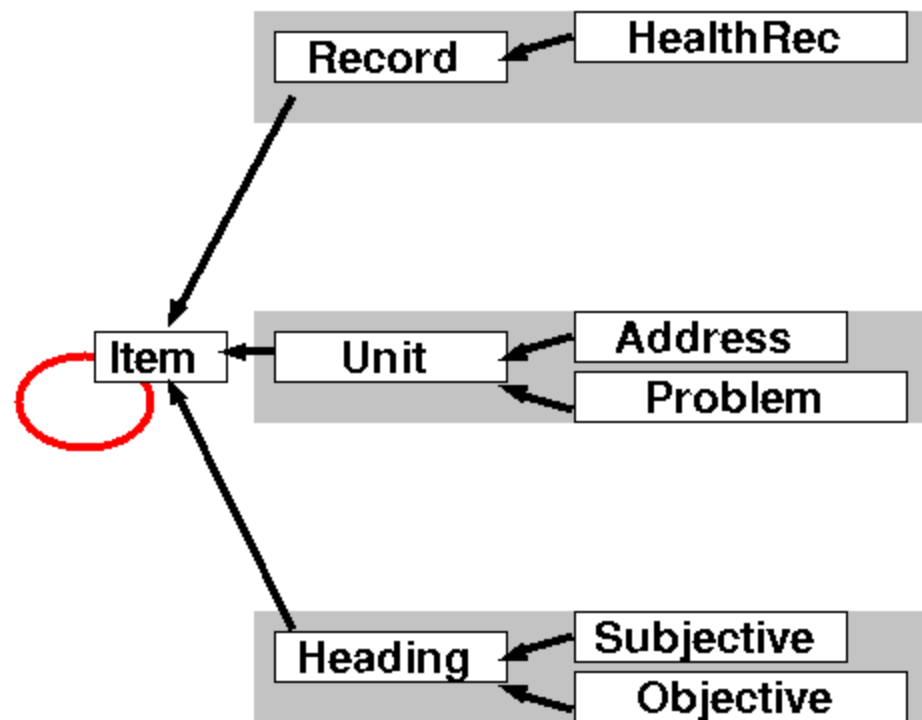
3.2 Hierarchy



Meta Level:

- Super Super type of all classes in the framework is Item
- Item is a container and has set/get/remove methods
- no set/get methods in subclasses

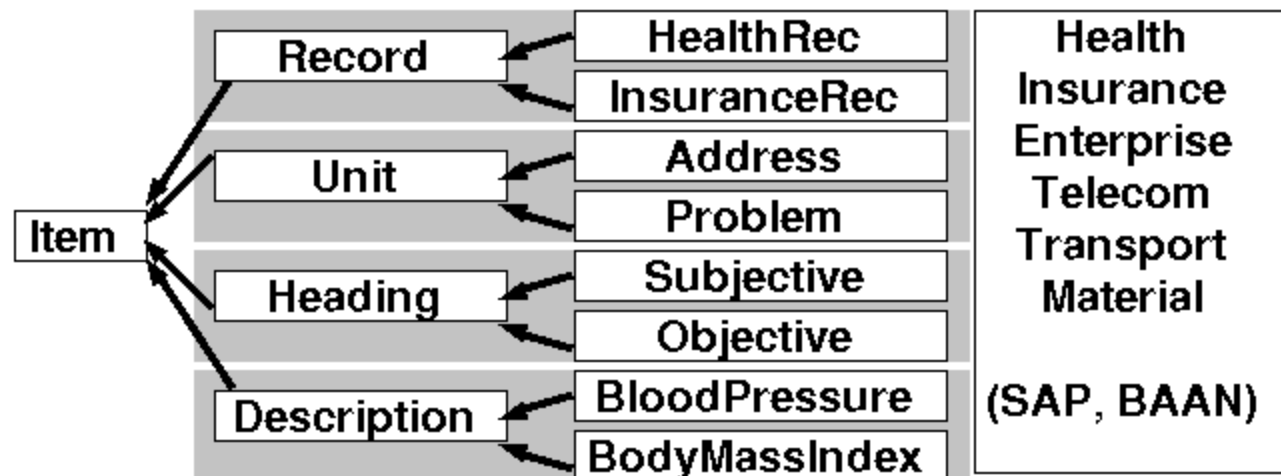
3.2 Hierarchy



Characteristic:

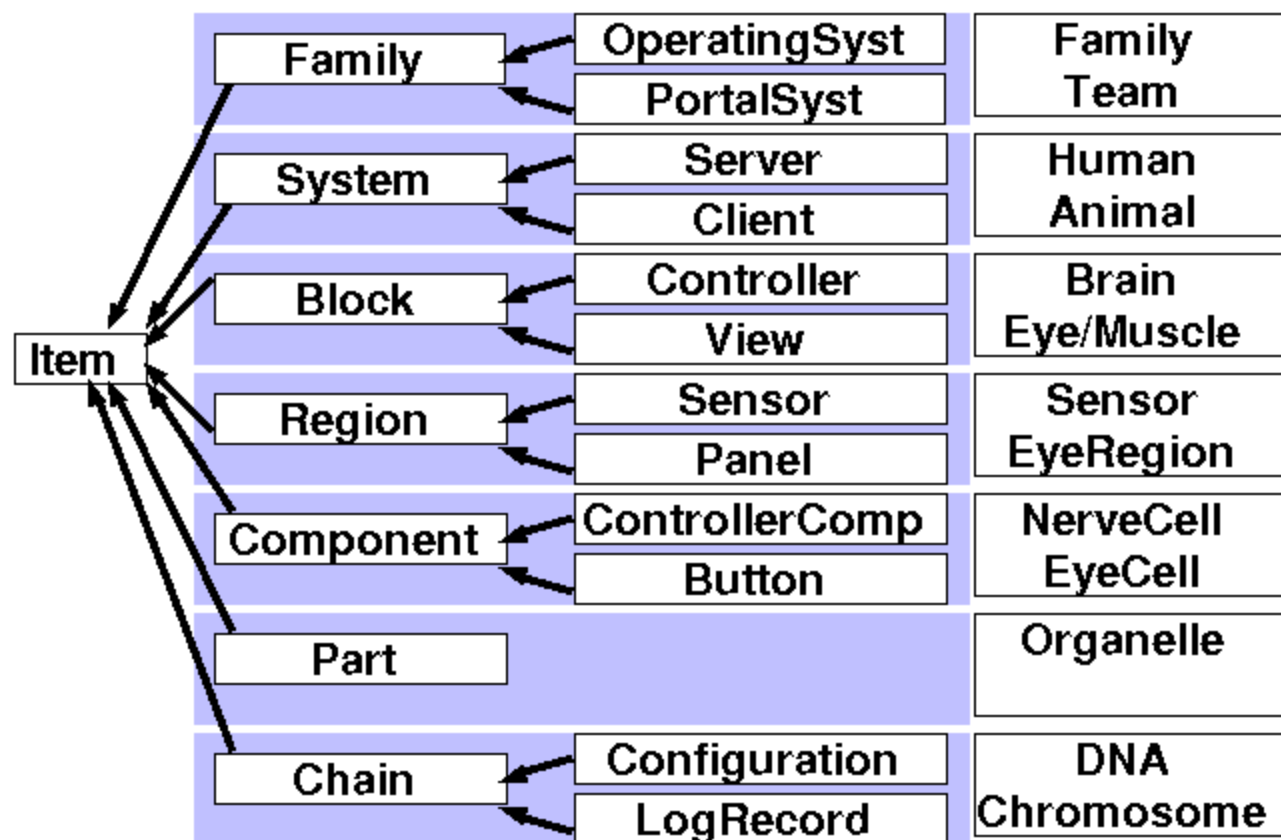
- every Item in Universe is hierarchical
- Urelement as root of this tree (Big Bang?)
- infinite Micro-/Macrocosm

3.3 Ontology: Model

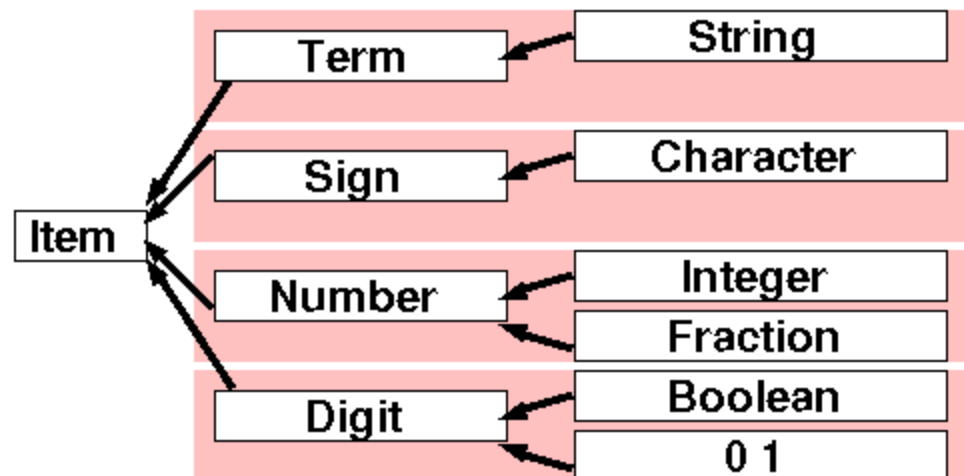


Initial idea:
OpenEHR/GEHR

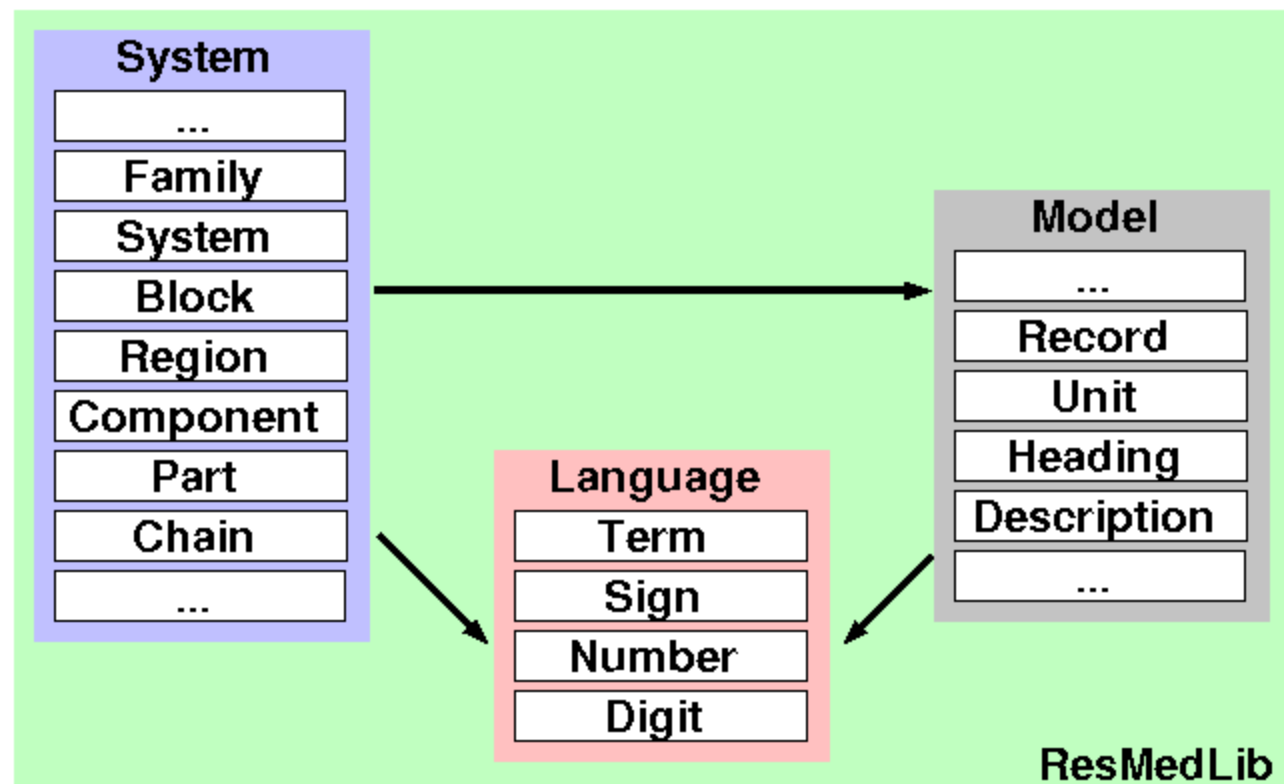
3.3 Ontology: System



3.3 Ontology: Language



3.3 Ontology: ResMedLib



3.4 Lifecycle: Apache

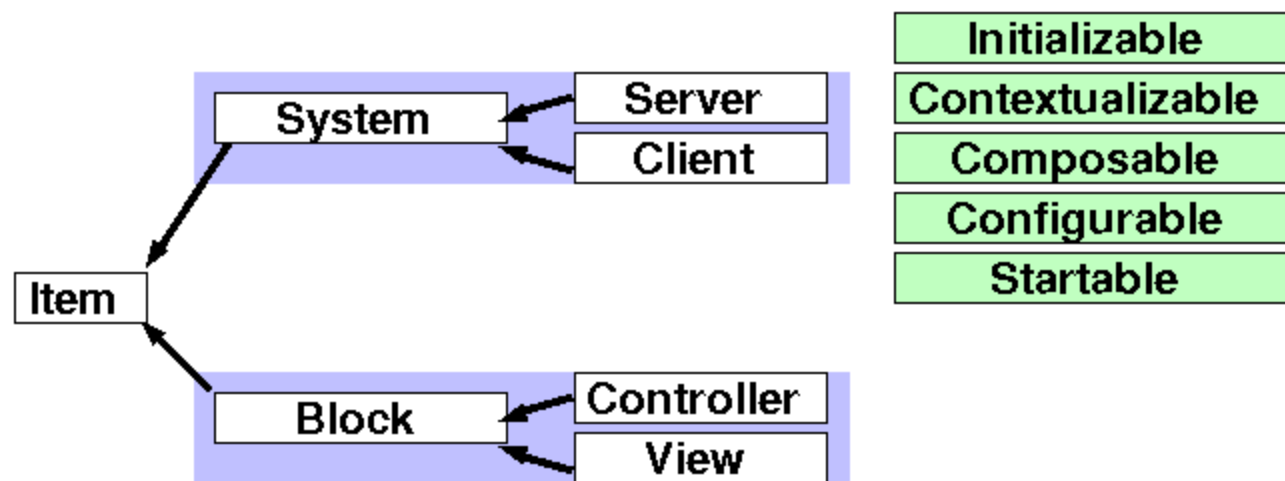
- Humans are a Tree, Universe is a Tree, everything is a Tree
- running a system: **startup** until whole system tree is built
- exiting a system: **shutdown** every item of the system tree
- in between the system **lives** (endless loop waiting for user actions) and can be used to solve tasks

Lifecycle for Apache Jakarta Server:

- 
- constructor
 - contextualize
 - compose
 - configure
 - initialize
 - start

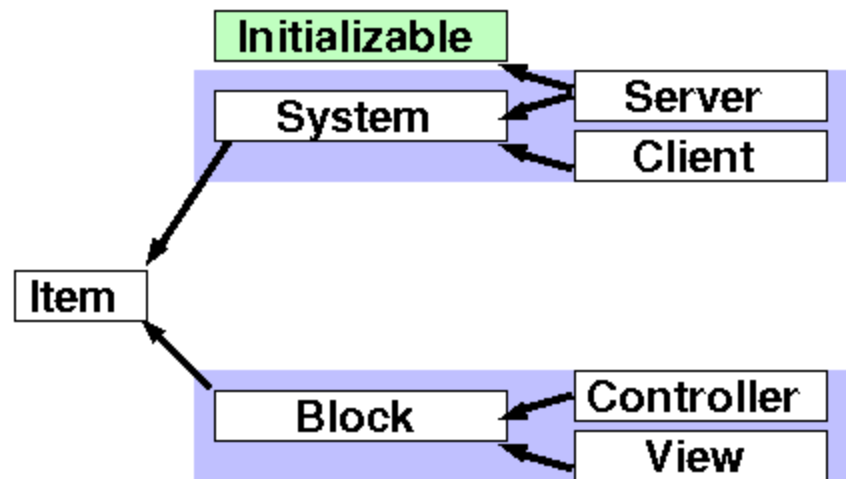
- 
- stop
 - finalize
 - deconfigure
 - decompose
 - destructor (not in Java)

3.4 Lifecycle: Concerns (Aspects)



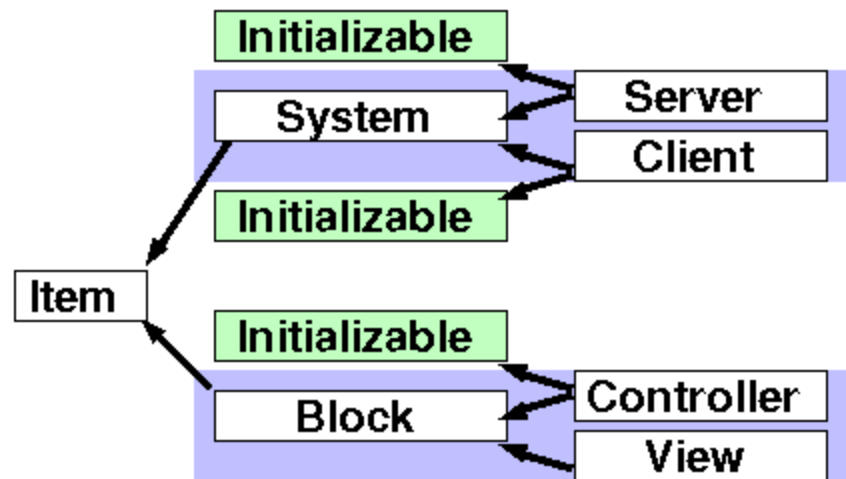
- one concern (interface) for each lifecycle method

3.4 Lifecycle: Concerns (Aspects)



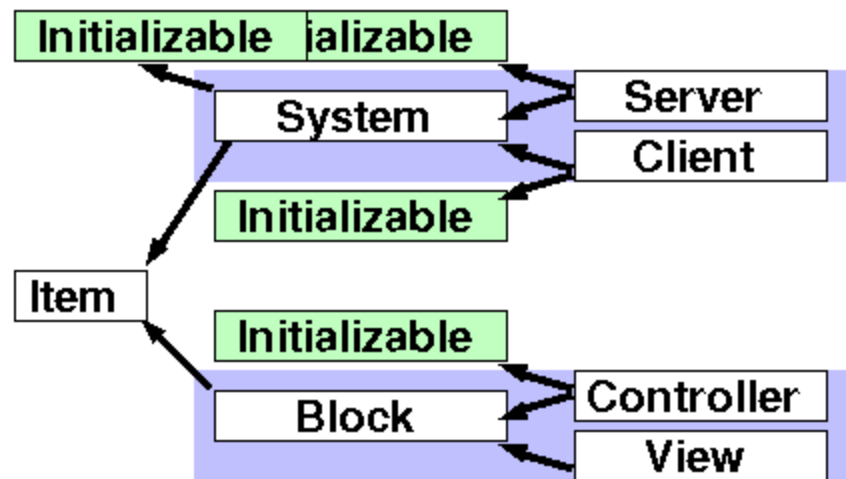
- classes inheriting from concern interface have to implement the corresponding lifecycle methods, e.g.:
 - `initialize()`
 - `finalize()`

3.4 Lifecycle: Concerns (Aspects)



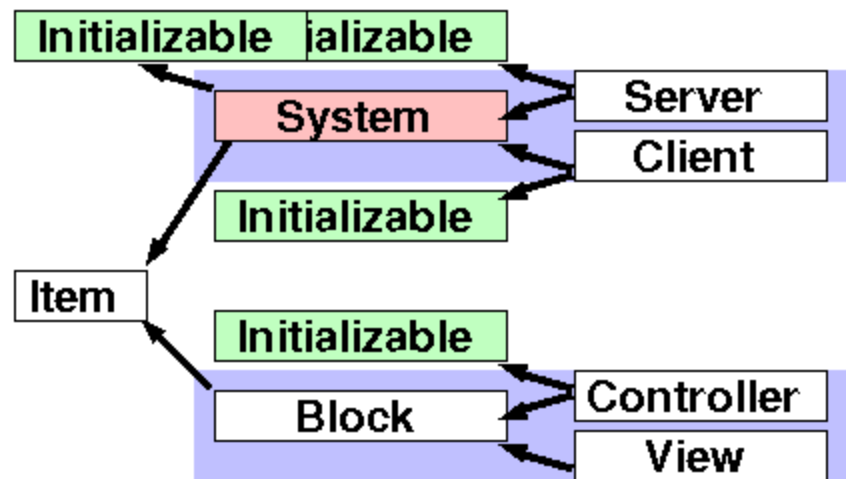
- **redundant code** cannot be avoided (many classes have to implement the same lifecycle methods again and again)

3.4 Lifecycle: Concerns (Aspects)



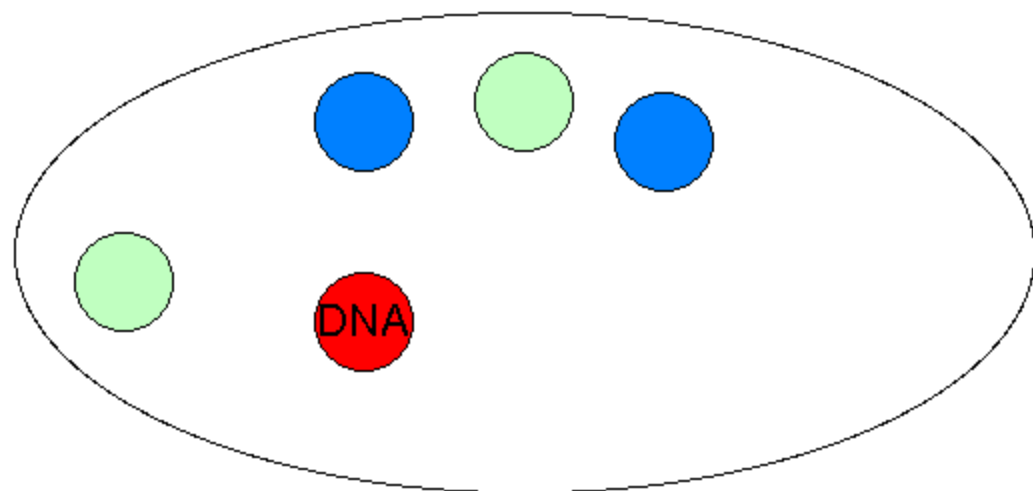
- **overlapping concerns** when parent and child classes inherit from the same interface

3.4 Lifecycle: Concerns (Aspects)



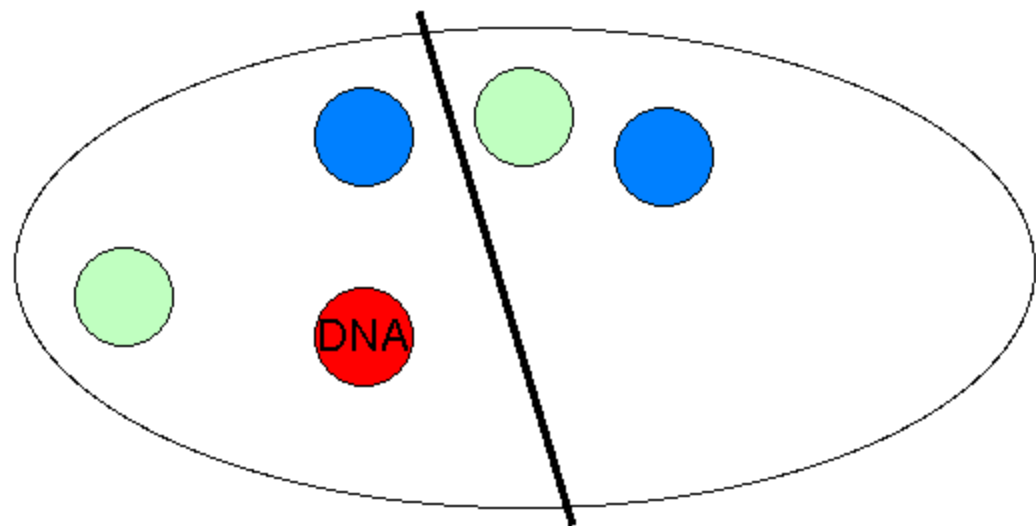
- instead, **LayerSupertypes** should implement functionality
- Concerns **spread** functionality, an Ontology **bunches** them
- Concerns are useless when having a proper Class Ontology

3.4 Lifecycle: Cell Separation



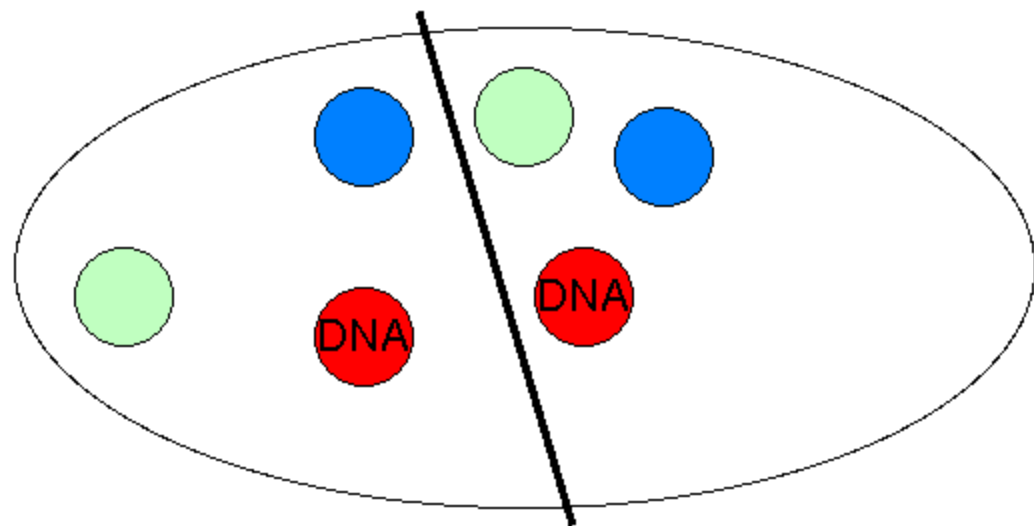
- cell core with DNA and chromosomes
- organelles such as chloroplasts, mitochondria, ribosomes
- cell plasma
- cell wall

3.4 Lifecycle: Cell Separation



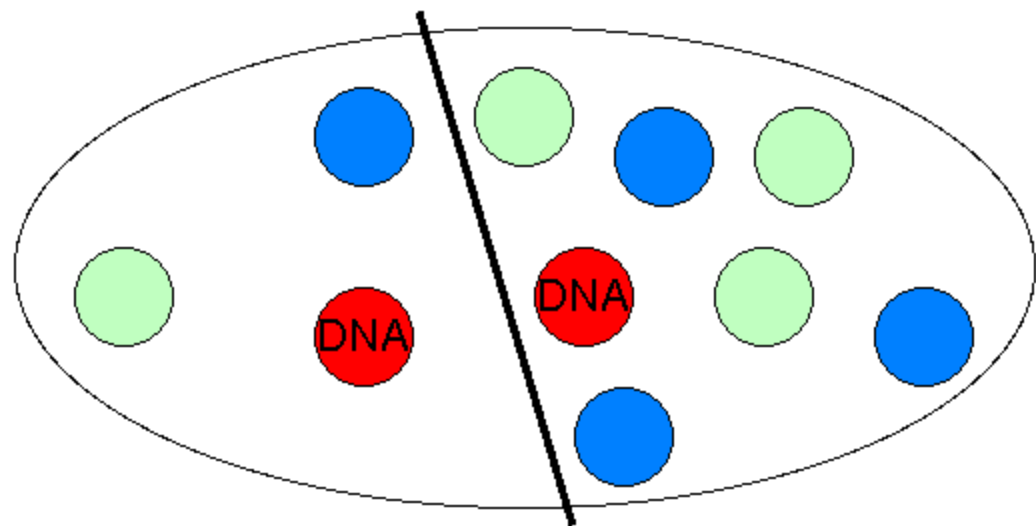
- constructor (cell separation)

3.4 Lifecycle: Cell Separation



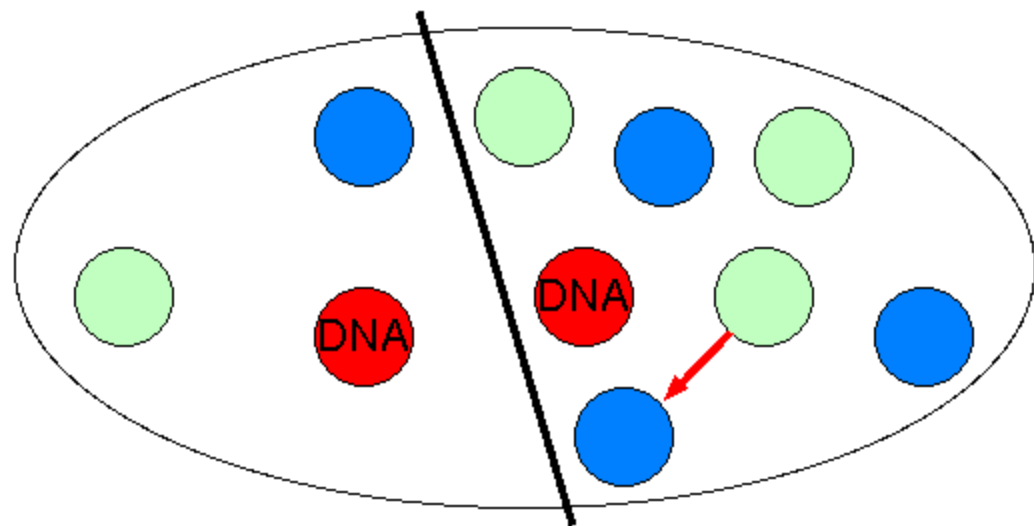
- constructor (cell separation)
- globalize/deglobalize (forward configuration = DNA, log record)

3.4 Lifecycle: Cell Separation



- constructor (cell separation)
- globalize/deglobalize (forward configuration = DNA, log record)
- contextualize
- initialize/finalize (create/destroy child items = organelles)

3.4 Lifecycle: Cell Separation



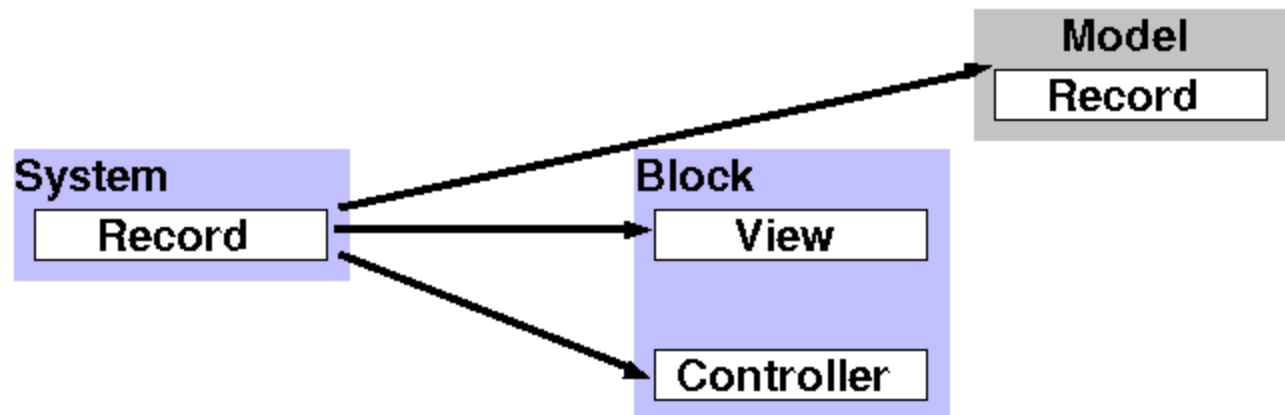
- constructor (cell separation)
- globalize/deglobalize (forward configuration = DNA, log record)
- contextualize
- initialize/finalize (create/destroy child items = organelles)
- **link/unlink** (associate child items = interaction)

3.5 Linking: MVC Example

System

Record

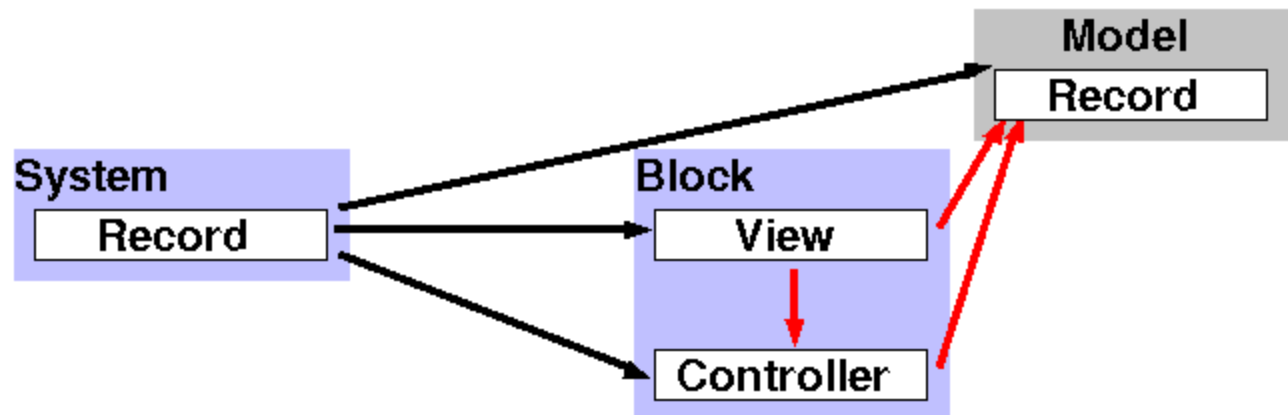
3.5 Linking: MVC Example



Hierarchical Composition →

- constructor (cell separation)
- globalize/deglobalize (forward configuration = DNA, log record)
- contextualize
- initialize/finalize (create/destroy child items = organelles)

3.5 Linking: MVC Example



Hierarchical Composition 

Loose Association 

- constructor (cell separation)
- globalize/deglobalize (forward configuration = DNA, log record)
- contextualize
- initialize/finalize (create/destroy child items = organelles)
- **link/unlink** (associate child items = interaction)

3.6 CYBOP Advantages

- **Clear Dependencies** between all objects (Class Ontology)
- Amalgamation of common (partly changed) **Design Patterns**
- No use of Interfaces/Concerns/Aspects (Layer Supertypes) --> **concentrate** all inheritances to one root class, **instead of spreading** functionality to many parents
- **Avoidance of abstract classes** (Framework useable as is, in every state; no parent class should rely on child class)
- Truly **independent Components** (Branches of System Tree)
- **No Garbage Collection** necessary (Lifecycle)
- **No static/Singleton** access to objects (global objects are forwarded in globalize method); if static: To whom belongs the static object? Who is responsible for its creation/destruction?
- **No synchronize** key word necessary if working in threads (proper instances of Ontology)

4 Res Medicinae

1 Introduction

2 Classical System Design

2.1 Physical Architecture

2.2 Logical Architecture

3 CYBOP

3.1 System Modelling

3.2 Hierarchy

3.3 Ontology

3.4 Lifecycle

3.5 Linking

3.6 Advantages

4 Res Medicinae

4.1 Aims

4.2 Analysis

4.3 Design

4.4 Implementation

4.5 Vision

5 Summary and Future

6 Appendix

6.1 Abbreviations

6.2 Sources

6.3 Tools



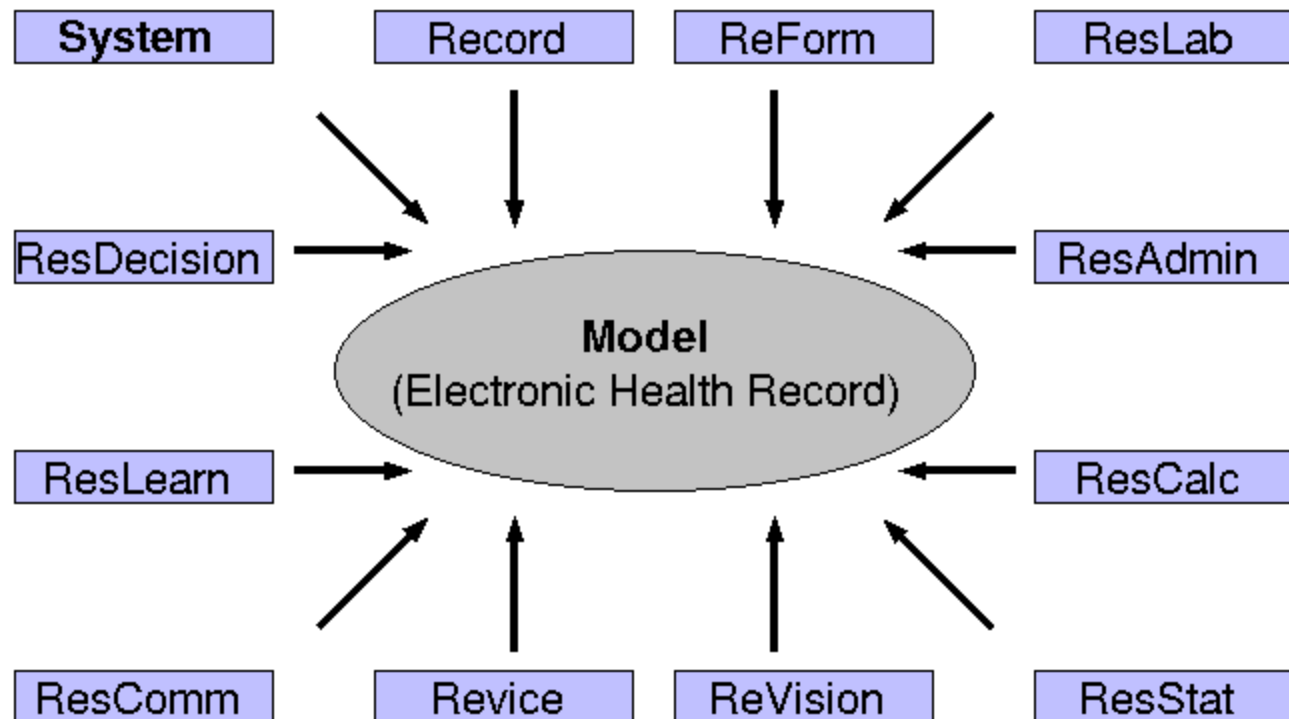
4.1 Aims

- **comprising** solution for Medicine
- **intuitive** use
- **latest technology**
- adhere to **common standards**, openness for other systems
- overcome high pricing in the realm of medical info systems
- **free, stable, secure, platform-independent, extensive**
- is and will be free in every meaning
- contributors enjoy working together
- encouraged by the idea of sharing their knowledge with those people living on *the poorer side* of the world

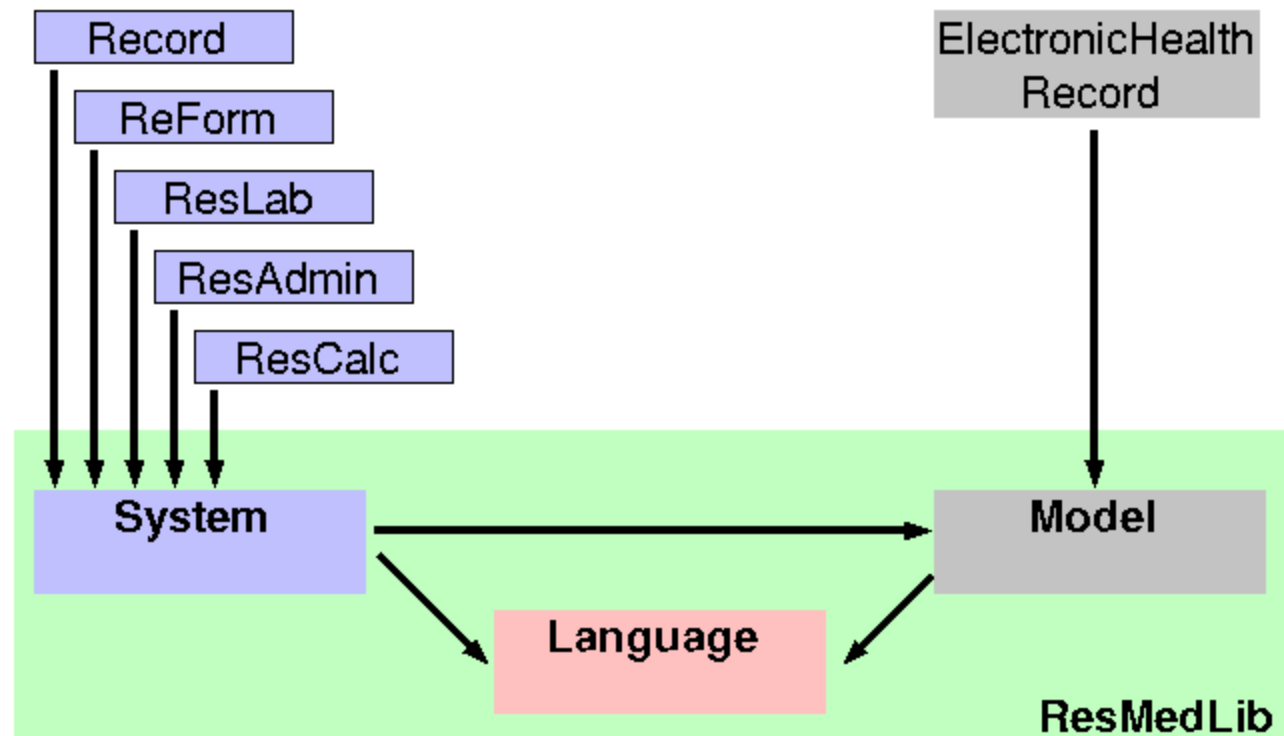
4.2 Analysis

- **Analysis Document**
 - grows fast, already quite complex
 - more than 100 discussion partners (50% doctors) in mailing list `resmedicinae-deutsch@lists.sourceforge.net` (German)
 - **International effort for a common document?**
- Identify **Model/Domain Objects** and their relations
 - Health Record (Person/Patient)
 - Problem, Contact
 - SOAP
- Specify **System/Application Components** (main use cases)
 - Medical Documentation (Health Record)
 - Form Printing (Report Generation)
 - Laboratory Data Retrieval
 - Billing

4.2 Analysis



4.3 Design



4.4 Implementation

- GNU GPL license
- Java JDK 1.4
- basic framework with GUI capabilities
- first modules: Record, ReForm, ResAdmin (diploma works)



ResMedLib (GUI, Persistence, Communication) →

Systems (ResMedPortal, Record, ReForm) →

Models (Healthcare, Insurance) →

4.5 Vision

- 2007 Medical Doctors in their practice
- 2007 Patients to access personal EHR/update data
- 2008 Laboratories
- 2009 Pharmaceutical companies provide drug databases
- 2009 Pharmacies
- 2010 Hospitals (doctors, training, administration, materials)
- 2011 Public Health Offices
- 2012 Health Insurances provide automatic billing
- 2014 Hardware companies

5 Summary and Future

1 Introduction

2 Classical System Design

2.1 Physical Architecture

2.2 Logical Architecture

3 CYBOP

3.1 System Modelling

3.2 Hierarchy

3.3 Ontology

3.4 Lifecycle

3.5 Linking

3.6 Advantages

4 Res Medicinae

4.1 Aims

4.2 Analysis

4.3 Design

4.4 Implementation

4.5 Vision

5 Summary and Future

6 Appendix

6.1 Abbreviations

6.2 Sources

6.3 Tools

5 Summary and Future



- Every item in universe is and belongs to a **Hierarchy** (Tree)
- Hierarchies (**Ontologies**) apply well for Information Systems
- Three major ontologies: **Language, Model, System**
- Design systems like **Human Beings**
- Build common (Domain) **Model Ontology** for business areas
- Items live in **Lifecycles** (similar to cell separation in biology)
- **No** Dr. Frankenstein! **No** theoretical bubbles, all from practice!

- **ResMedLib** encapsulates common functionality to allow easy implementation of Modules (Systems/Applications)
- **ResMedicinae** is still in early stage but already contains some module prototypes
- **Software Development** is not in crisis. It just starts to become interesting!

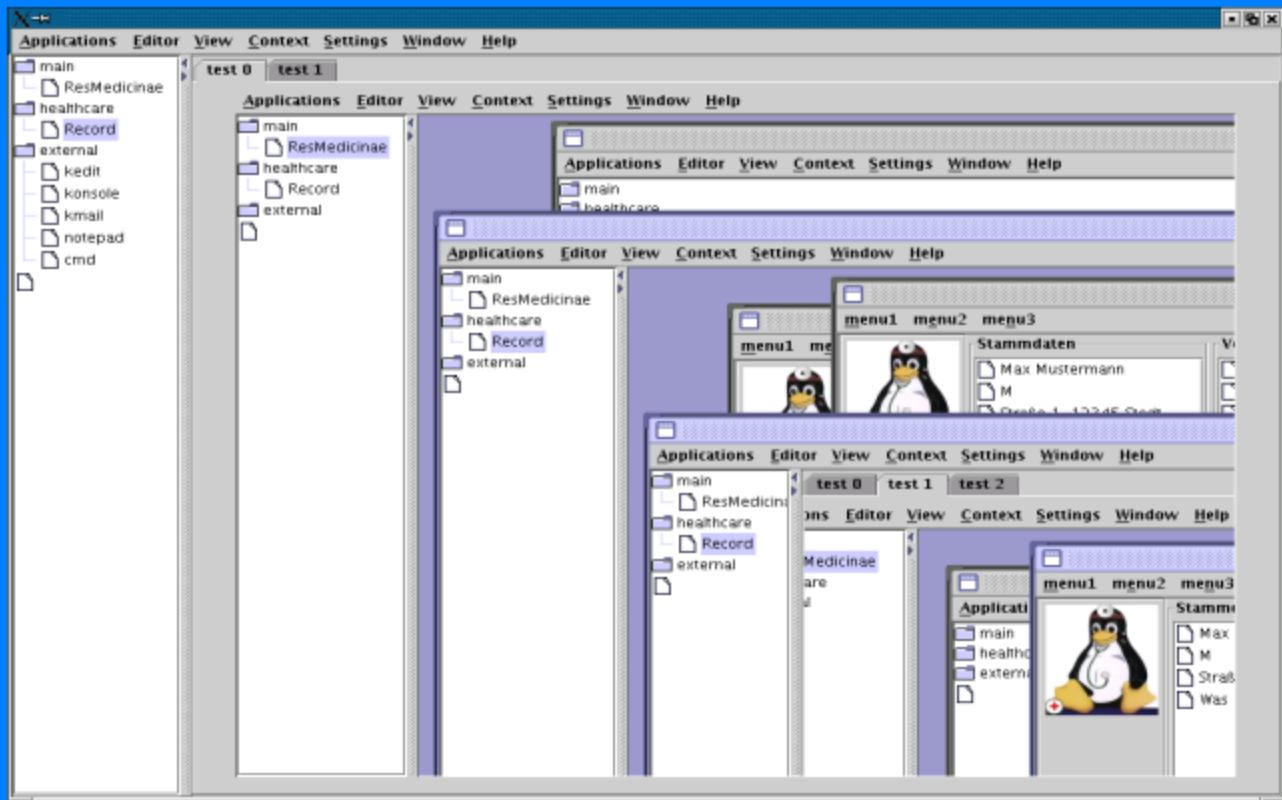
Thank you!



Questions?



ResMedPortal: Cascading



Record: TreeTable

Res Medicinae

System Patient Problem Fenster Episode Image Marker Einstellungen Hilfe

Back

Patient

Jens, Bohl, male, 24.02.1978

Festeburgstrasse 102, 98749, Steinheid

A rh+, 69kg, 178cm,

DAK

CAVE

Penicillin Allergie

Probleme

Gastritis

Medikation

TreeTable

TextEditor

GraphChart

Topology

Filter

All

New Problem

New episode

New contact

New medication

Remove problem

Remove episode

Remove contact

Remove medication

Probleme

Electronic Health Record

Problem

Episode

Partial Contact

Objective

Subjective

Assessment

Plan

Medication

Dose

Partial Contact

Probleme	1	2	3
Jens, Bohl, 24.02.1978			
Gastritis		erosive Gastritis nach rezessivem Ulkus	
Mon Oct 07 14:06:42 CEST 2002(First)		Mon Oct 07 15:28:17 CEST 2002(Last)	
Mon Oct 07 15:28:17 CEST 2002			
Zungenbelag, positiver Bois-Druckpunkt			
Starke Schmerzen im Oberbauch, Völlegefühl			
Reizmagen, Stress, Hyperaktivität			
Gastro-Duodenoskopie			
Omeprazol	300mg		
1-1-1			
Mon Oct 07 14:06:42 CEST 2002			

Neues Episoden-Template angelegt.

Record: Topological Doc

Res Medicinæ

System Patient Problem Episode Image Marker Einstellungen Hilfe

Back

Patient

Name, Vorname Versicherung CAVE Probleme Medikamente Anamnese

Geburtstag Geschlecht Nummer

Adresse Typ

Telefon

Befundung

Zurück

Beschreibung
lat. Systema
Skeletale

Aktuelle Region
Name
Brustkorb
Ziel
Brustkorb dorsal

Vorschau



TreeTable TextEditor GraphChart Topology

ReForm: Prescription

ReForm

File Card index Patient Formular Help

Choose patient

Select patient

Patient name:

Billert, Ronny
Bohl, Jens
Hager, Steffen
Koehler, Marcel
Krause, Susanne
Kunze, Torsten
Meier, Jochen
Mudler, Sybille

Remove Select Close

Change patient data

Name: Kunze
First Name: Torsten
Birth: 25.04.1977
Street: Max-Planck-Ring 6d
Zip Code: 98893
Town: Ilmenau
Country: Germany
Insurance: BARMER

Change Clear all Close

New patient

Name: Welsch
First Name: Christina
Birth: 10.10.1982
Street:
Zip Code:
Town:
Country:
Insurance: AOK

Add New Close

Formular: Script

Krankenkasse bzw. Kostenträger
BARMER

Hrb. Impf-Spr.-St.
Bsp. mittel-alt. Senior
0 2 3 9

geb. am
Kunze, Torsten 25.04.1977

Kassen-Nr. Versicherten-Nr. Status

Vertragsarzt-Nr. VK gültig bis Datum

Rp. (Bitte Lesartensymbole eintragen)

Vertragsstempel

Unterschrift des Arztes
Muster 16 (4.2002)

Bei Arbeitsunfall auszufüllen

Unfalltag Unfallbetrieb oder Arbeitgebernummer

6 Appendix

1 Introduction

2 Classical System Design

2.1 Physical Architecture

2.2 Logical Architecture

3 CYBOP

3.1 System Modelling

3.2 Hierarchy

3.3 Ontology

3.4 Lifecycle

3.5 Linking

3.6 Advantages

4 Res Medicinae

4.1 Aims

4.2 Analysis

4.3 Design

4.4 Implementation

4.5 Vision

5 Summary and Future

6 Appendix

6.1 Abbreviations

6.2 Sources

6.3 Tools

6.1 Abbreviations

API - Application Programmer's Interface

C/S - Client/Server

CICS - Customer Information Control System

CORBA - Common Object Request Broker Architecture

DB - Database

DDL - Data Definition Language

EHR/EMR/EPR - Electronic Health/Medical/Patient Record

ERM - Entity Relationship Model

GEHR - Good Electronic Health Record

GUI - Graphical User Interface

HDTF - Healthcare Domain Task Force (formerly CORBAmed)

HL7 - Health Level 7

HMVC - Hierarchical Model View Controller (design pattern)

6.1 Abbreviations

HTTP - Hypertext Transfer Protocol

IBM - Industrial Business Machines, Inc.

IDL - Interface Definition Language (CORBA)

IT - Information Technology

JDBC - Java Database Connectivity

JMS - Java Messaging Service

JSP - Java Server Pages

KDE - K Desktop Environment

OOP - Object Oriented Programming

OODBMS - Object Oriented Database Management System

OSS - Open Source Software

PL/1 - Programming Language for old host systems (procedural)

Qt - C++ library of Trolltech, base library of KDE

6.1 Abbreviations

RDBMS - Relational Database Management System

RMI - Remote Method Invocation

RPC - Remote Procedure Call

SOAP - Simple Object Access Protocol (XML based)

SQL - Structured Query Language

TABEX - Table Management System of BOI company

TUI - Technical University of Ilmenau

UML - Unified Modelling Language (diagrams)

VSAM - Virtual Storage Access Method (old hierarchical DB)

XML - Extended Markup Language

6.2 Sources

- Apache Jakarta Framework <http://jakarta.apache.org>
- Martin Fowler. Analysis Patterns. München: Addison-Wesley, 1997.
<http://www.martinfowler.com>
- Erich Gamma et al. Design Patterns. München: Addison-Wesley, 1996.
- GNUmed. <http://www.gnumed.org>
- Hierarchical Model-View-Controller (HMVC).
http://www.javaworld.com/javaworld/jw-07-2000/jw-0721-hmvc_p.html
- K Desktop Environment (KDE). Some images and icons.
<http://www.kde.org>
- Open Infrastructure for Outcomes (OIO). Resource Library.
<http://www.txoutcome.org/scripts/zope/library>
- OpenEHR/GEHR. <http://www.openehr.org> <http://www.gehr.org>
- OpenEMed. <http://www.openemed.org>
- Res Medicinae. <http://www.resmedicinae.org>
- Scope. <http://scope.sourceforge.net>

6.3 Tools

- [Linux](http://www.linux.org) <http://www.linux.org>
- [XFree86](http://www.xfree86.org) <http://www.xfree86.org>
- [KDE](http://www.kde.org) <http://www.kde.org>
- [KPresenter](http://www.kde.org/koffice) <http://www.kde.org/koffice>
- [Gimp](http://www.gimp.org) <http://www.gimp.org>
- [ArgoUML](http://www.argouml.org) <http://www.argouml.org>
- [jEdit](http://www.jedit.org) <http://www.jedit.org>
- [Kate](http://www.kde.org) <http://www.kde.org>
- [Konqueror](http://www.kde.org) <http://www.kde.org>
- [Cervisia](http://www.kde.org) <http://www.kde.org>

6.4 Why Java?

- Modern, object oriented language, simple to program
- Offers complex, well designed class library
- Mix of compiled and interpreted language
- Slower than C++, faster than Python

Problem: Sun license, but:

- Sun has opened Java code
- Sun supports Open Source Community (Star/Open Office)
- Hope that Sun opens Java completely in the future
- Free GNU GPL Java is in development
- Dependencies from JDK can be minimized with framework
- Projects can then easily be ported to other languages
- Great popularity among students and developers

6.4 Java Alternatives

C++ (System language)

- free, fast in execution
- difficult to program, errors caused by dead pointers, OO only as late extension

Python (Script language)

- free, fast coding, compilation not necessary, can be used as *glue* for linking components
- typeless, slower than Java, not for building complex systems

Other (VisualBasic, Tcl/Tk, PERL, Pascal etc.)

- smaller number of features
- to little weight for market importance